Building AI-enabled .NET Applications

Integrating AI into .NET Apps

Understanding the Basic Concepts of AI

How cool would it be if your .NET applications are not just responding to user inputs, but predicting them, offering a solution to the user before the question is even asked? This is the power of AI, and it is not just some futuristic concept from the movies anymore. It is here, and it is changing the way we interact with technology. As developers, we're now steering this revolution. I am Gill Cleeren, CTO at Xabia Microsoft Services in Belgium. And in this course, we are going to see what is the potential of AI in your .NET applications. From integrating AI models with a few lines of code to creating intelligent and predictive systems, we'll explore how to build smarter apps or leveraging your .NET knowledge. Let's jump in and see how AI can take your .NET apps to the next level. AI, short for artificial intelligence, has quickly become a game-changer in modern software development. With AI, your applications can think, learn, and respond in ways that we couldn't imagine just a few years ago. But where do we start when it comes to incorporating AI into .NET applications? Well, that is what this course is all about. Throughout this course, we'll walk through the different approaches to integrating AI into .NET apps from using pretrained models through services like OpenAI and Azure to more customized solutions like the Semantic Kernel. By the end of this course, you'll have a strong understanding of the different ways you can AI-ify your applications. Even if that wasn't a word, it is one now. It shouldn't come as a surprise when I say that things around AI change. Well, often. That means that the SDKs and the libraries are also often changing. Sometimes even with breaking changes when the APIs of the underlying models change. To make things simple, when you want to follow along, please use the versions of the libraries that I'm using so things should work. I will regularly update this course when changes happen to make sure it stays relevant. But please let me know in the comments also if you encounter something that doesn't work any more, and I'll do my best to update things as quickly as possible. For years, computers have been following orders that we gave them. Process this data. Return this input. They were fast and reliable, but they weren't capable of thinking. That was our job as developers. Now, thanks to the advancements in AI, things have changed. Computers aren't just following instructions anymore. They're learning from them. They can adapt, improve, and even make decisions on their own. This evolution in computing is at the core of what AI brings to the table. In the next few modules, we'll explore how this new capability is changing how we build applications and how you can take advantage of this in your .NET apps. In the last couple of years, artificial intelligence has changed from something we see in the movies to being a part of

everyday software development. Why you may ask? Well, for one, because AI offers many benefits to the user. Imagine having applications that provide insights which were previously impossible or interacting with software using natural language using voice assistants or chatbots. AI is enabling us to build smarter applications that not only respond to users but also adapt and learn from them. Before we get hands on, which is what this course will do, I'm going to introduce you to some of the key concepts around AI. Let's start with defining a few key terms that will come up throughout this course. AI, or artificial intelligence, is simply the ability for a machine to mimic human intelligence. Then, there is generative AI. This is a branch of AI that is focused on creating new content like text or images based on patterns in the data. You'll also hear me mention LLMs, or large language models, quite often as we use these a lot in this course. These models have been trained on a huge amounts of data and are capable of understanding and generating human-like text. So how do LLMs actually work? At their core, large language models are a type of artificial neural network, which is basically a collection of numbers, also known as parameters, that are all connected to each other. Think of it as being similar to how our brain works. Our brain has neurons or brain cells, which are interconnected, and AI models mimic this idea but with numbers. Now AI models don't understand words, images, or sounds like we do. They only work with numbers. When you give the model some input like sentence, it gets converted into these numbers. Those numbers are then processed by the neural network. Based on how those connections between the numbers are set , a new set of numbers comes out on the other end. After the model processes those numbers, it converts them back into something that we can understand, like text or images. An LLM can have billions or even trillions of numbers, by the way, which is why we call them large. So, what exactly is happening when an AI model generates text? Let's say that you type once upon a. What the large language model will do is predict what comes next by guessing the next word based on patterns it has learned from lots and lots of examples. It sees a sequence, once upon a, and thinks okay, what's the most likely word to come next? Maybe time. Once it predicts that, it goes further once upon a time in and so on. This process continues as long as you keep feeding it input or until it runs out of context. It is not actually writing a story like a human would, but rather making the best guess based on the data it has seen during training. So you made me thinking, how do LLMs then learn? Well, the process is similar to how humans learn. It just starts with training. Just like how we learn from reading, observing, and experiencing things, LLMs are trained on vast amounts of text, millions of books, articles, and websites. During training, the model makes predictions about what comes next. And if it's wrong, it adjusts itself to get better. That's actually something we should perhaps also do more. Anyway, the numbers inside the model, often referred to as the parameters, are constantly updated. Every time the model processes new data or gets feedback, it fine-tunes these numbers to improve predictions in the future. Have we as humans then also involved? Absolutely. After the model generates a result, we, as users or developers, can provide feedback on whether it was good or not. If the model's

output is incorrect or doesn't make sense, humans can come in and give feedback, helping the model understand how to improve. This collaboration between the model's internal updates and external human feedback is what makes LLMs progressively smarter and more accurate over time. The most famous model you've probably used is ChatGPT. And it made the GPT abbreviation about as common as the word Google today. GPT stands for generative pretrained transformer. Hm, can't say that would have been the first thing I would have thought of. Let's dig a bit deeper. Generative refers to the model's ability to generate new content, whether that's text, code, or even images based on the input it receives. This is exactly what we saw earlier with the once upon a time example. Remember? The model is generating words that come next based on what it has learned. Pretrained is a very crucial here. This means that the model isn't learning from scratch every time you use it. It has been trained on a massive dataset beforehand. So it already has a deeper understanding of language patterns and language structures. The training we discussed earlier where the model learns by processing text and updating its numbers all happens during this pretraining part. And finally, transformer refers to the specific type of neural network architecture that powers GPT. This architecture is what allows the model to handle large amounts of text and process it in parallel, making it incredibly efficient at predicting the next word, sentence, or even entire paragraph. When it comes to AI models, we have options and these are constantly changing. So what you see here is a snapshot from when this course was created in late 2024. GPT-4 and GPT 3.5 are widely used models from OpenAI created to handle tasks like text generation. Then there's Gemini from Google, Claude from Anthropic, Llama from Meta, and of course, well, there are many more. Each of these models has different strengths, whether it's speed, cost, or specialization. Some run locally, and some integrate with cloud platforms. Choosing the right model depends on your specific needs. And it is not a goal of this course to cover that. AI models come in all shapes and sizes. You have text-to-text models that can take a piece of text and transform it. That can be translating, summarizing, or even generating a new piece of content. Then there's text-to-image, text-to-video, image-to-text, and even speech-to-text models. Multimodal models are hot right now, and they combine inputs and outputs across these different models. And the inference says describes an image in text and have the model generate a detailed caption. Multimodal is becoming the norm nowadays. AI has grown far beyond doing simple things. Today's AI models can do much more than just generate text or recognize images. They can write code, act as personal coaches, and even provide medical advice. As developers, this opens up a world of possibilities for building applications that are interactive, adaptive, and truly intelligent. Now, when we work with a model or we think we do, we aren't directly engaging with it when we're using something like ChatGPT. ChatGPT is actually a product just like Copilot, which communicates with the model behind the scenes. It is this app that manages things like the message history. Since the underlying model is stateless, it doesn't know anything about past questions or interactions. The

functionality of the model is exposed to an API, and it is this API that the products will connect to. Through this API, we as developers can also interact with the model. We can build apps that will go through this API endpoint and thus offer AI experiences to our app users. When we talk about interacting with AI models, so via the API, we're really talking about prompts. A prompt is key in the whole interaction between us the human and the model, and it is what you provide to the model. The quality of the output depends on the quality of the prompt. For example, asking the model to summarize this article might give you a decent result. But if I refine that prompt to summarize the key points of this article about .NET development and focus on what has been added in the latest version, it will give you a much better response. We'll be writing and using quite a few prompts, and you'll see that the results we are getting back are very heavily influenced on what we're sending in to the prompt. Let us by means of example break down what makes a good prompt. A vague prompt like summarize an article will like to give you a general, non specific result. A better prompt would be summarize the key points of this article about AI. And an even better one would go further. The more specific you are, the better the results you'll get from the model. We've now touched on some of the main concepts of AI. There are many terms related to AI and AI development, and we'll touch on quite a few more throughout this course. I think at this point you're familiar with at least the basics to be able to follow along. And by looking at all the terminology used here, it is obvious we'll be learning a lot more in the coming modules. So, what can you expect from this course? It is a hands-on practical guide to integrating AI with your .NET applications. We're not here for the theory. Instead, I'll show you how to use the OpenAI .NET API library, Azure AI services, and Microsoft's Semantic Kernel in real-world scenarios. We'll be adding intelligent features to your apps very quickly, and you'll see how easy these libraries make it for us, even without any AI knowledge. This course is for developers who are new to AI, but also eager to get started using it in a meaningful way beyond the nice demos you've all seen. And we'll do all this with .NET. Equally important, let me also be clear what this course is not. We won't be diving deep into AI theory or building AI models from scratch. That is a whole different story. This course is focused on practical application, how to take existing AI models and use them effectively in .NET. Like I said, you don't need to be a machine learning expert to follow along. If you have a good foundation in .NET, you're halfway there. Let's keep things practical and focused on building awesome AI-powered applications.

Using the Different Approaches for AI in .NET

Now, let us switch gears and start our journey into using .NET for AI development. Maybe the first question that pops into your mind is why would we use .NET for AI? Isn't Python a more obvious choice? Well, great question. If we think about it, .NET is an incredibly versatile platform. Whether you're building server-side apps, cloud-based systems, or desktop applications, .NET has you

covered. And it comes with powerful tooling like Visual Studio that you are probably familiar with. So the familiarity is a big plus here. You can build on what you know in .NET to now incorporate AI into your existing apps and workloads. .NET itself also comes with capabilities built in like ML.NET and the ONNX runtime, which we won't even be covering in this course. And don't forget, the tools that we have like Visual Studio come packed with AI-assisted code generation through Copilot. What is possible when we combine AI with .NET? Well, to say it with the cliche, the sky is the limit. You can build agents and chatbots that respond to users in real time, create computer vision applications that analyze images, or even develop applications that interact with customers using synthesized voices. AI can also help with classification, predictive analysis, and so much more. Those are the topics we'll be focusing on in this course. You've just seen that, typically, models will come with an API that we can use to interact with it, and those are typically REST APIs. Integrating AI into your .NET apps can therefore be done in the usual .NET way using the HttpClient. This approach will always work, and we'll see how this can be done. However, creating the HTTP request manually is sometimes tedious work and can also be pretty complex. This interacting with models is often more than just a single call from the app to the model API. Therefore, SDKs are created for some models, and the good news is that those are, again, plain .NET SDK, making it extremely easy to work with models from .NET. These SDKs wrap the underlying HTTP calls, therefore giving us typed access to interact with the models. The models from OpenAI come with such an SDK, acting as a wrapper around the OpenAI API, giving you access to models like GPT for tasks like text generation, translation, and even code completion. This library is built on the open API specification from the OpenAI's API. Try repeating that sentence a few times. It's a fun challenge. We'll start very soon with exploring and using this SDK. Next is Azure OpenAI service, which is a managed service that brings the power of Open AI models to the enterprise. With Azure OpenAI, you'll get all the capabilities of the OpenAI models, but with the added benefits of Microsoft security, privacy, and enterprise features. These are really the strengths of this product. From a .NET developer's perspective, interacting with OpenAI or with the Azure OpenAI service is done using the same SDK. So what you'll learn here in this goal applies to both. Now we won't be diving in the specifics of Azure OpenAI service in this course. To make it simple in terms of naming, there's also the Azure AI Services suite, so not to be mixed with the Azure OpenAI service. This is a suite of tools for AI and machine learning interactions, covering everything from vision to speech, language and decision-making. These prebuilt APIs make it easy to add advanced AI functionality to your apps without needing to build anything from scratch. There are SDKs for nearly all services, and we'll use quite a few of these in this course to enhance our .NET applications. Finally, let's talk about the Semantic Kernel, a lightweight open-source SDK from Microsoft designed to make managing multiple AI models easier. It helps you orchestrate and integrate various AI models into your .NET apps, streamlining the development process. This tool is especially useful when working with multiple AI services, and I

will introduce you to the Semantic Kernel in this course as well.
Being overwhelmed with choices is something I find to be common when working and learning about AI in general. For what I've just explained, you probably have this impression too. I therefore made a choice to include in this course targeted at you the .NET developer, the OpenAI SDK, some Azure AI Services SDKs, and the Semantic Kernel. The latter is only touched on since there is also a dedicated course on Semantic Kernel on Pluralsight, but more on that later.

Storing Keys in Your Application Securely

One more thing, but an important one. When working with AI services in general, one important aspect to consider is security, specifically how to store your API keys securely. These keys authenticate your app with the AI service, and they need to be protected. They are issued by the service and also help the API to track usage and manage access. Quite a few of these services are paid for. And through this API key, the service also knows how much you're using it. So, it's a billing thing too. Often, you will pay for access to the API, and any request that you send to it is deducted from your credit, typically the amount of tokens which is used. Tokens, excuse me? No worries. We'll come to that later. For now, see it as the number of words used in the request and the response that you're getting back from the model. There are different approaches to working with keys securely without any risks of leaking anything when checking in code. Environment variables are one option. The Secrets Manager in Visual Studio is another one. Or you can use Azure Key Vault, a popular service to store and retrieve secrets in your app, both during development, as well as for production.

Demo: Storing Keys in Your Application Securely

Let me show you in this brief demo how I will handle secrets. Let's assume we want to store a secret, an API key for now, in an ASP.NET Core Blazor application. Typically, since this is a setting, it's stored as plain text in the appsettings.json or perhaps in the appsettings.development.json. Now while that may be our initial thought, that is not a great idea since this API key should be protected. And if we store it in plain text, it will be checked in and visible for everyone to use. Not so great, right? So a better approach will be to store this in the user secrets. I can right-click here on the project file and select Manage user secrets. A file will be created on the local machine where I can now safely store my secret values. I'll make the ModelSecrets. And in there, I have the API key and its values. That is now only visible on my local machine; it is not getting checked in. If we look in the project file, we can indeed see that this line here has been added. It contains the link to the local file containing the secrets. And these settings are read out by adding this line here in the Program.cs. Also, app settings

and environment-specific app settings are read out. Any identical ones will be overwritten. I also add AddEnvironmentVariables, which will ensure that, for example, Azure App Service settings are also being read out. In this case, I can, for example, store the real API key in there too. And since this is the last entry here, that value will be used then. Next, I'm also using the IOptions pattern here using this line. From configuration, the ModelSecrets section will be read out into an instance of ModelSecrets. Of course, I have added here in my code that mimics the structure of the secrets in my secrets file, but now in a typed way. Now in code, we can get an instance of this ModelSecrets plugged in through dependency injection, and that's what I'm doing here using IOptions in ModelSecrets and instances injected containing the settings. And I can then get the actual value through this line here. If we run this now, we can indeed see that the value is read out and shown on the screen, in this case from my local secrets. I am not making that value visible to anyone else, which is what we are after here.

Using OpenAI from .NET Apps

Introducing OpenAI and Its Models

OpenAI has given the world some amazing models to generate text and images. While these are great tools to use standalone through ChatGPT's interface, the functionality of the OpenAI models can also be integrated in our own .NET applications. This will usually improve the experience that you can offer to your users. Seeing and learning how we can do this is exactly what we will be doing next. Since all of us are pretty new to this AI thing, let's take a moment to get familiar with OpenAI itself and the models they have created so far. They have blown us away with models like GPT-4 and DALL-E, which are used for a wide range of tasks, including generating human-like text and creating amazing images from a piece of text. These models are designed to work with simple APIs, which means you do not need to be an AI expert to use them. OpenAI is a company focused on working towards safe and highly capable artificial general intelligence, also known as AGI. Excuse me, you said AGI; it's AI. Well, sorry, I actually did mean AGI here. What is that all about then? Well, you may have heard the term AGI, or artificial general intelligence, being thrown around in conversations about AI. AGI refers to a more advanced form of AI that can perform any intellectual task a human can do. It is AI that can learn and adapt in multiple areas, not just a specific task like answering questions or generating images. While AGI is still the end goal for a lot of AI researchers, including the people at OpenAI, we're not quite there yet. What we do have today are AI models that are good at particular tasks like language processing or image generation, but they don't have that broader human-like intelligence. So for now, we'll be focusing on how we can use specialized AI models to accomplish some pretty amazing things within our .NET applications. This in itself

is already a giant step forward. So why should you care about using OpenAI in your .NET applications? Well, integrating OpenAI unlocks many possibilities for us to add in our apps. First, these models are really, really powerful. You can easily add natural language understanding, content generation, and conversational capabilities to your apps without a need to build and train your own AI models from scratch. Secondly, OpenAI provides pretrained models, meaning they have already been trained on massive amounts of data. So they're ready to use out of the box. You don't need to be an expert on AI. Plus, OpenAI has integration with Azure, making it easy to incorporate it in your existing Azure infrastructure and services. OpenAI has created some great models. First, we have GPT, a model designed for natural language processing. It can generate human-like text based on the input you give it. It is also what powers tools like ChatGPT. Whether you needed to answer questions, generate creative text writing, or assist with customer service tasks, which we will be getting with Bethany's Pie Shop, GPT is the default model. Next, there's DALL-E, the model responsible for turning text descriptions into images. You can give it a prompt like a sunset over the mountains or a futuristic city, and it will generate a custom image for you. And then there's Whisper, which is used for speech recognition. If your application involves voice commands or converting text to speech, Whisper will be extremely useful.

Working with the OpenAI API

All right. Now that we know what OpenAI is, let us get started with interacting with the models from our apps. OpenAI provides a simple API that makes it easy to access their models from any platform, including .NET. The API allows you to send an HTTP request, usually a piece of text or a prompt, and then receive a response from the model. Whether you're asking GPT to complete a sentence or generate an image using DALL-E, everything is done in the end through API requests. The nice thing about this approach is that it is platform-agnostic, meaning you can use the same API whether you're building a web app, a mobile app, or desktop application using .NET, Java, or any other language or platform. It is, after all, just the REST API. OpenAI has created the platform.openai.com website, which is OpenAI's central hub where you can explore the documentation of the APIs for the different models. On this location too, you can manage your API keys, track your usage, and there's also a playground where you can experiment with the models before integrating them into your application. The playground is a great tool. It allows you to test different prompts, tweak the parameters, and see how the models respond in real time. It's essentially a sandbox where you can try out different IDEs before writing any code. Plus, because OpenAI uses the pay-as-you-go model, the portal will also show you how many requests you've made and how much it is costing. For the API access. which we will need in this course, you'll need to at least temporarily have a paid subscription on the API.

Demo: Using platform.openai.com

OpenAI has a nicely maintained site where we can find all the documentation for interacting with its APIs. and it is available from platform.openai.com. Now let us explore this site a bit more in this demo. Now I am not going to take you through all what is on here. You can look at this at your own pace. But I'll instead highlight what I think is important for us right now. Here, under the Capabilities item, we can see a list of the most important features that OpenAI offers at the time of the creation of this course. Of course, text generation is here at the top. It is the most common way that we interact with LLMs today. And so that is probably the most commonly used endpoint. Anyway, in here, we will find some general information of text generation and a quickstart for all the capabilities listed here. We can also get some more information and some examples. I am more interested in the endpoints here since that is what we will be using from our own application to interact with the OpenAI API. Here we get a nice description of how we can indeed interact with the OpenAI API. We see here a sample curl request that we can execute, so an HTTP request that we can send. So if you want to interact with OpenAI, we'll need to send in a request that has this particular structure. Of course, sending in a request is only half of the equation. We also need to work with the response. And to do so, it will be necessary that we know what we're getting back to the response format. That is shown here as well. This we can then later on pass in our own applications. One thing that I skipped here is the API key, the OPENAI_AI_API key part here. And to make this request, we'll need to include our own API key. As you probably have heard, AI uses a lot of energy and resources, and it's therefore not free to execute requests on the API. You'll need to buy some credits in order to be able to use it. I've already signed up, so I could just log in here. You'll need to do that as well. Once logged in, you need to click here on the Settings wheel. And then under Billing, you can select how you want to pay. What you're paying here are actually credits for the API calls, and this is separate from having access to the paid ChatGPT. Even if you already have the latter, you still need to pay for credits for the API. As you can see, I have a few dollars left in here. Once those are gone, I need to recharge in order to be able to continue using the API. With the credits added to my account, we can go to the API keys. And in there, I can create an API key, which we already saw we needed to add in our HTTP requests. We can click here on Create new secret key and can give it a name. We won't link it with the project here, and we can select all permissions here. Here we go. Now we have an API key created, which we can then use within our applications to pass along with the requests. By including this API key, OpenAI can see if we still have access to credits to make this call. So it's both a security thing and a billing thing. But this key should remain a secret. We already saw how we could store secrets in our application securely during development. As it also says, don't include this key inside client code since it may be exploited by others, therefore draining your credits. I will start using this API key very soon.

Making Manual HTTP Requests to OpenAI's API

To interact with the OpenAI API, we need to authenticate each request, and that is where those beforementioned API keys come in. When you create a paid subscription to OpenAI's API access, you can create an API key, like we saw in the demo. Every time your application makes a request, you include the key in the authorization header as bearer and then your API key. This ensures that OpenAI knows who is making the request and allows it to track your usage and bill you accordingly. It's a simple but secure way to authenticate API calls. Interacting with OpenAI's API is just a matter of sending a request. In this case, we're making a POST request to OpenAI's chat/completions endpoint. Our goal here is to ask gpt-4o to explain artificial intelligence in simple terms. The model we're using is set to gpt-4o, and we're limiting the response to 100 tokens. You'll notice that we're also setting parameters like the temperature, which controls how creative or random the response is. The lower the temperature, the more deterministic the response will be. Once the request is sent, the model processes it and sends back a JSON object containing the generated text. I've then just mentioned again the word tokens that I've mentioned a few times already. Now, what do I mean with tokens? Well, let me explain the concept as simple as possible since, to be honest, it is good to know, but you don't really need to know the nitty details in order to use AI from .NET. So tokens are the building blocks of how large language models process language. So then, what exactly is a token? In simple terms, a token can be a word, a part of a word, or even just a character depending on the language model being used. The model breaks down any input text into these smaller chunks or tokens, which allows it to understand and generate language more effectively. Think of it as a way for the model to simplify the language and focus on the meaningful pieces of the text. For example, if you type in Bethany's Pie Shop has the best apple pies, the model might break this down into the following tokens. Beth any 's Pie Shop has the best apple and pies. Each of these tokens represents a piece of information the model can work with. Sometimes it might even break down a single word into multiple tokens by splitting Bethany's into multiple parts, Beth any and s. Most AI models, especially when using services like OpenAI, charge based on the number of tokens processed. So understanding how your input is tokenized can help you control the cost and optimize the performance of your AI solutions. Plus, models also have a token limit, which determines how much information you can send to the model at once.

Demo: Making Manual HTTP Requests to OpenAI's API

All right, we now know that the functionality of OpenAI is available over an API, a REST API that is. So let's now use that knowledge to make a request to OpenAI and work with the response all from plain .NET and C# code. I'm going to use,

let's say, the manual approach first. So I'm going to make the HTTP request myself and parse out that response. I have a class here, ConnectWithHttp, that has this run method, which is going to hand over the control to this CallGpt4Api. I am passing in the prompt, the plain sentence that contains, in this case, the question for the model to generate a short story about a pie shop, something that we could perhaps use for the about page of our website. So in here, we're going to use the HttpClient, a well-known built-in class from .NET for making HTTP requests and getting back responses. At least, hopefully. We set the API key in the header using the Authorization header field with the bearer token. So we can authenticate our API requests with OpenAI. The next step is constructing the request body. The structure I'm using here, I got from the platform.openai.com website where we already encountered this structure. Notice that we're using the GPP-4 model, and we're specifying two messages. First, we have a system message that frames the context. Here, we define that the assistant is a helpful assistant. This helps guide the tone and the behavior of the AI. The second message is the user prompt where we pass in the prompt from the string from earlier asking GPT-4 to create a short story. I'm also passing in some options. The max_tokens here is set to 200. This controls the maximum length of the response from the model. In this case, we're limiting it to 200 tokens where a token can be as short as one character or as long as a whole word. The more tokens we use, the more you pay. You actually pay for tokens. I also set the temperature to 0.7. This parameter controls the randomness of the AI's output. A low temperature, so closer to 0, makes the response more deterministic and focused, while a higher temperature closer to 1 makes the output more creative and random. Once the request body is ready, we serialize it into JSON and create a StringContent object to send it over the wire. Next, I'm creating the actual HTTP POST request to the OpenAI's chat chat/completions endpoint. If the request is successful, then I deserialize the JSON response here and I extract the generated text from GPT-4. This response is stored in the jsonResponse.choices[0].message.content. That is where GPT-4 returns its output. And we log that to the console. Let's now run this and see things in action. I have fast-forwarded a bit since it will take a few seconds for the model to come up with this story. But here we go. Here is the result, which will be different every time we run this application. So, working with OpenAI's APIs is pretty straightforward for a .NET developer, but it is quite a lot of hassle. There has to be a better way.

Using the OpenAI .NET Library

We can use the HTTP approach, and that is simple to do. Although using it for more complex endpoints tends to be a bit more of a hassle, but it will work fine, all based on the HttpClient class as we have seen. But let's make things even simpler by using the OpenAI .NET API library. This library is a wrapper around the API that makes it easier to work with from a .NET application. It's available as a

NuGet package that we can add to our apps. Instead of manually constructing HTTP requests, the .NET library provides a clean, object-oriented interface for interacting with OpenAI's models. This will help us to avoid a lot of the boilerplate code and make our code more readable and maintainable, so without worrying about the low-level stuff. The library also fully supports async, which is the default when making underlying API calls. So, as said, the code we have in our apps will become a lot easier. We'll go from code that looks like this where we are manually setting up the API request to code like this where we are now using some types brought in by the SDK. We'll understand the code together in just a minute. For now, I want you to understand that it is a lot easier and more compact using the SDK. The first thing we need to do is bring in the OpenAI .NET library, you can easily install it using NuGet as said. And once that is done, it is just a matter of setting up your API key and initializing the client. Don't forget about what we saw about safely storing that API key. In the snippet on this slide here, we're using gpt-4o to build a basic chat feature. We're creating a ChatClient instance and passing the user's input. The model processes the input and returns a response. This simple interaction shows how easy it is to integrate natural language understanding into your .NET application. The library abstracts a lot of the complexity. So all we need to do is focus on the user's input and how we want to handle the model's response. It's as simple as that, almost too simple if you ask me. We can directly instantiate the ChatClient calls like I did in the previous slide, but we can alternatively also start from the OpenAIClient class that I'm instantiating here in the first line. And then I'm using the GetChatClient on that, passing in, again, the model name I want to use. Using the OpenAIClient class is an often-used alternative, but it can be used to give us access to instances for use with the different types of model.

Demo: Using the OpenAI .NET Library

Let us now see how we are going to use the SDK. And before we do that, we need to set up a few things, namely bringing in the correct NuGet packages and also bring in the API key. And so once we have that, we'll start to use the API to create some very simple chat interactions. Now for this and the next demos, we will also be using a console application, and the different functionalities we look at will all be in different methods. Before we look at these, let's take a look at the project file. And in there, we can see that I have a reference to the SDK package to the OpenAI .NET API library. As said, I also need an API key. We saw earlier that we should store this in a safe location, which can't be leaked to the client side. Now, of course, this is a console application, which runs on the client anyway. So storing the API key in a secrets.json file is not very useful, right? I've therefore decided to make things simple in this console application. And so I have stored the key inside the launchSettings.json file as an environment variable. We can now from code easily fetch this value. Alternatively, we could have also stored this value in Windows in the path. But let's keep it simple and store it here as part

of the application itself. Now we should go to the ChatCompletions class. In there, in this first method, SimpleChat, the goal is to send the prompt to an AI model and get back a response. We'll pass in which model, which OpenAI model I should say, we want to use. At first, we create an instance of the ChatClient class. This is a type that comes with the OpenAI .NET API library, and it serves as our interface to communicate with the chat/completions endpoint. This endpoint is the one that is used to generate all sorts of text. A ChatClient is initialized with two parameters, the model name, which specifies which OpenAI model we want to use, like GPT-4 or GPT 3.5, and the API key. Next, we are calling the CompleteChat method on the ChatClient instance. Now this method is responsible for sending a prompt. And that is, in this case, the message say Hello AI world to the OpenAI model and then returning a ChatCompletion object. The ChatCompletion class encapsulates the result of the chat, which includes the response generated by the AI, of course. And what we're seeing here is very important. This method is abstracting away the complexity of manually creating the network request. Internally, it handles sending the HTTP request to OpenAI's API. So it sends the input message and waits for the response to come back. Finally, we then show the message in the console. I can run this from my Program.cs. I also need to pass in the model name. The latter I am getting from config, so the appsettings.json, and those settings are read out through these lines here in the appsettings.json file. We can see which model I'm using, which is gpt-4o. As I run this, and yes, it's a very simple prompt. So we're getting back the same text but now returned by GPT. Let's look at this next method, which looks similar, but there is a difference. Instead of directly instantiating the ChatClient as we did in the previous example, we're now starting with an OpenAI client. Now this is a more general purpose client that serves as an entry point for interacting with various OpenAI services. You'll notice it is initialized with the same API, also retrieved from the environment just like before. Now the key difference here is that the OpenAI client can provide access to different functionalities, so not just chat. It is like a parent class for managing a broader set of API interactions. After initializing the OpenAI client, we use it to retrieve a ChatClient by calling GetChatClient and passing in the model name. Once we have that, the code is the same. When we run it, of course, the output is the same as well. Now I've used the synchronous versions here. But since we are essentially talking with APIs, we can and yes, maybe, we should talk using asynchronous code. Many of the methods here also have an async version, which is what I'm using here. Now although the code is now asynchronous, if we run things again, we still need to wait for the response to come back as a whole. So, we don't see anything while the model is working on creating the response.

Demo: Using Streaming Completions and History

In the example we just went through, we made a simple request to the GPT model, and the entire response came back as a single chunk. Now while this

works fine, there are cases where we might want to get the response as it is being generated, especially for longer outputs. This is where the streaming version of the client comes in. Instead of waiting for the full response, which we just had to do, we can start receiving data as the model generates it, which improves the responsiveness of your app and creates a smoother user experience. It feels more interactive and engaging and gives users the same experience as they have with ChatGPT, for example. If you want to use the streaming version, we start by sending the chat messages to the CompleteChatStreamingAsync method. This method returns an AsyncCollectionResult of StreamingChatCompletionUpdate, which gives us access to the ongoing stream of updates. Then, using the await foreach, we iterate over each completion update as it arrives. Inside that loop, we further break down the completion update to extract individual contentPart elements. For each part, we output the text to the console in real time. We'll start this demo by looking at how we can start getting in the response from the model while it is being produced. That is how the model generates the response typically as well. You can think of it as the model basically writing out the response for you. What we want is seeing this in our apps, just like in ChatGPT. So the code for this is what we have here. And of course, I'll take you through that step by step. We're again using the ChatClient. That hasn't changed. Now notice I am using the CompleteChatStreamingAsync method instead of the CompleteChat method, which I used before in a synchronous or asynchronous way. And what this does is beginning a streaming response for the passed-in prompt. The response here will also trigger a large response. So we'll actually see it coming back. The method will return an AsyncCollectionResult of StreamingChatCompletionUpdate objects. So essentially, this allows us to receive the AI's response in chunks. So it's streaming it back as it becomes available, avoiding that we have to wait until the entire message becomes available, which is, in fact, what we had so far, and then use an await for each loop to asynchronously iterate over each StreamingChatCompletionUpdate object as it arrives. And for each update, we loop through the content update, which contains ChatMessageContentPart objects. Each ChatMessageContentPart represents a piece of the response, and we print out a text in real time using the Console.Write here. Let's look at this in action, I'm going to run it now. And yes, we can see that the response is streaming in from the API as it becomes available. Now there is an important concept to understand how these AI models work. They are stateless. This means that each time we send in a prompt, the model has no memory of previous interactions. It processes each request in isolation without any awareness of what was said earlier. In other words, the completion you get is just based on the input that you provide in the prompt. But this works fine for simple text completions. But for a true chat experience where you want the AI to remember the context of the conversation, we need to implement history support. And to achieve this, we must not send just the latest prompt but the entire conversation history with each request. This way, the model can contain continuity in its responses

and start a real back-and-forth dialogue. So to turn this simple interaction into a real chat system, we'll have to handle the history ourselves, passing in previous messages along with a new one to provide the necessary context for each response. That is what I'm doing here in this next method in essence. I'm introducing a message history. The difference here is how we create the prompt. Instead of sending a single message, we're building a conversation history using an array of ChatMessage objects. So, that is this one here. These messages simulate the flow of a conversation by capturing both the user's inputs and the assistant's previous responses. I have four messages in this simulated history so far. This first one is a SystemChatMessage to set the behavior of the assistant, telling it that it is a knowledgeable helper in the food space. Next, this is a real user chat message where the user asks for help. Then we add an assistant chat message simulating the AI's response where the AI warmly welcomes the user. And then I add another user chat message where the user asked for a list of 10 popular sweet pies from around the world. By including this message history, we're providing the AI with context so it can generate a more informed response based on everything that has been said so far. Next, we use the CompleteChatStreamingAsync, so that's similar as before. But this time, we're passing in the entire array of chat messages instead of just a single prompt. This enables the model to consider all previous messages as part of the current conversation. So it can maintain continuity in its responses. The code that follows is identical. It will stream the response. If we run this now, we'll see a response that is indeed taking into account the entire message history.

Demo: Optimizing the Prompts

The way we communicate with the model to express what we want it to create is just plain language, your native tongue even. And it is what is referred to as the prompt. In the previous demos, we didn't spend too much time on the prompt just yet. However, the way you set it up can and will have a tremendous impact on the response that the model will generate. In this first sample here, I am setting up the message history, creating a persona that explains to the model how it should behave, so which role it should be taking on. That will already steer the results in the right direction, and it is something you'll often do to get started with. Then, the input of the user is added, and that is sent to the model. If we run this and we enter the name of a pie or cheesecake, we'll get back, well, a pretty decent response. And the response might be formatted entirely different than next time we run this since we haven't specified anything about what we are expecting in terms of format, the type of response, the type of words we're looking for, where the text will be used like in a cookbook or as promotional text on the site. However, we can definitely do that. Here's another example. We have now provided this extra context through a prompt that contains an example. The goal of this one, as the name gives away, is creating a product name, a fancier product name, for which I give an example or two. If the user enters a

cheesecake, come up with names like what you see here, and the same for cherry pie. If we run this sample, we'll see indeed that the name that has been generated follows the format of the example that I have provided. So by making the prompt more extensive, the results will also be more predictable and in the direction we want them to be. Here's another version. We have now extended the system prompt as we can see. I've specified the type of wording that you use, and I've also asked it to include emojis if possible. If I run this, we will indeed see a different type of outcome. Indeed, quite some fancy words have been used this time. And finally, in this example, I've asked it to create an ingredient table in a specific format using the metric system for quantities. I have also asked it to create a list in two languages, English and Dutch. The result, let's see. Well, there we go, contains indeed a nicely formatted table. In fact, two like we asked, each containing the ingredients per language. So there we go. The prompts will really influence the result, as we can see with these examples. Going further, you will see that it will even create much longer and more detailed prompts as well.

Demo: Changing the Model Parameters

We can pass in options for the model. By default, so far, we've relied on preconfigured defaults. We can do this by creating chat completion options, which is what I'm creating here. They allow us to specify various parameters that affect how the model behaves. The MaxOutputTokenCount sets the limit for how long the response can be. In this case, I am capping it to 300 tokens, which ensures that the AI's reply doesn't get too lengthy. Tokens, as mentioned before, are like the currency. So the more tokens, the more money it will cost you. It's therefore often a good idea to limit this if you're not expecting a long response. Next, the temperature is set to 1, which makes the output more creative and varied. A higher temperature introduces more randomness. So the AI is less likely to repeat itself and more likely to explore, well, different possibilities. Lower temperatures make it more deterministic and focused. The frequency penalty and the presence penalty options are both set to 0, which means that the AI won't be penalized for repeating itself or reintroducing topics. These penalties are useful when you want to discourage repetitive or redundant responses But in this case, we're leaving them off to allow more free form creativity. If you run this, we'll see a rather short response that is indeed quite free form. If I change things a bit and I set the temperature lower to 0.5 and this time I add another option, namely that I want the response to be in JSON, we will see a different result. Doing this requires that we also change the prompt as it needs to be specified that the response should be in JSON format too. If we look at the result, we'll see a more strictly controlled response, which is also in JSON format. So indeed, to conclude, we have quite a lot of ways to control what the model is creating for us.

Demo: Creating an Open-ended Chat

In this demo, I want to take a few things together we have covered already and create a real chatbot that continues accepting new requests. And it will also take into account the message history, so the context, when creating its next response. And there won't be anything really new in this sample though, but let's walk through it together. And before we do that, let's run things first and ask it a first question. My question will be create a fancy name for a blueberry cheesecake. And then we get a nice first suggestion. And we can enter a new question, like do another one, and it will know that I've asked before about the blueberry cheesecake. So it will give me another suggestion. So we're building on the history. I'll now do one more and I'll ask it to do the same but now for an apple pie. And there we go. We get another name suggestion, again indicating that the history was included. In the code, I'm using a prompt that we already used before using the SystemChatMessage to create a persona and then a message that will be displayed too. Then I create a loop that will run until the input is empty. I add the message to the history. And then, again, using the CompleteChatStreamingAsync, passing in the messages, I'm streaming the response from the model. That's exactly what we saw in the running demo as well.

Demo: Generating Images Using DALL·E

Let us now shift our focus from text generation to image generation using OpenAI's DALL-E model. This model is capable of turning text description into images, which I think is also really cool. The process is quite similar to working with GPT, except instead of generating text, we're now quite logically generating images. In this example, will create an ImageClient, which is the type used for this interaction, and define parameters using an ImageGenerationOptions instance. Here we can specify things like the size of the image and the style. Once the request is sent, DALL-E generates an image based on the prompt we give it, and we can display that in our application or save that locally. And we'll see that in the demo. Image generation is the default of DALL-E, so text to image. However, the model is capable of more, like creating image variations. In this scenario, we upload an image and ask it to generate, well, a variation. The model can also be used to perform image edits. So let's now see how I can put image generation to work using DALL-E. I have added DALL-E 3 in the appsettings, so that is the model that I use first. Let's look at the code for this. And most of it will actually look rather familiar. We interact with an OpenAI client, pass a prompt, and receive a result. But of course, we're generating a visual representation instead of a textual response. So I'm using the ImageClient, which is specialized for handling image-related tasks with OpenAI. And it also needs a model name and an API key. It can be the same key, by the way, that we used also for text generation. The next key element is the prompt again. And this time, it is

quite a lengthy one, as you can see here, providing a detailed description that the image model DALL-E will use to create a visual. The prompt describes a cozy pie shop with a blend of rustic charm and modern design. I have, in fact, not created this myself. I've used a text generation model to create this prompt for me. Next, I'm passing in ImageGenerationOptions, allowing us to customize how the image will be generated. I set the image quality to high, ensuring that the output has a refined detailed look. I'm then picking one of the available sizes, and this is the largest one at this point. The style is set to Vivid and showing that the colors and the contrast really jump out of the image. Finally, the response format is set to Bytes, meaning that the image will be returned in a binary format we can then save to disk. Then we call the GenerateImage method, passing in the prompt and options. And the result is a generated image object, which contains the image data in binary format. These bytes I'm then saving to disk. So let's run this and see the result. Of course, this will take a few moments to generate. The image was generated in the bin/debug folder. Let's take a look at the one here. Wow, this really looks nice. It corresponds to the specs that I have provided in the prompt. And next to generating images from a prompt, DALL-E can also create image edits and image variations. At the time of the recording of this course, this could only be done using DALL-E 2. There is definitely a difference in quality noticeable between images generated with the different models. Both variations and edits are, from an API perspective, very similar. So let's take a look at image variations. I have in my existing application this image here of a waterfall. I can create an instance of imageVariationOptions and then, on the ImageClient, call the GenerateImageVariation method, passing in the original image and the options. I've let things run, and let's take a look at the result. t has created indeed an image similar to our original image.

Demo: Using Vision

Next to generating images, OpenAI models can also be given an image to describe. And I'll show that here in this vision-based demo. In the DescribeImage method which we see here, I'm going to ask the text generation model again to describe an image that I'm including as part of the message history. Here, I'm reading in the image, and we can see that I'm including this image as part of the message history. I'm using the CreateImagePart method to upload the image to the model. Next, using the regular CompleteChat method, we're asking the model to return to us the description of the image. Let's try this and see the result. So there we go. That looks like a very detailed description of a waterfall image. Think of this when you are generating, for example, alt tag descriptions for images on web pages. The model can create this automatically for you.

Understanding Function Calling

One of the limitations of language models like GPT is that they don't know everything. For example, if you ask the model for real-time data or specific information that changes frequently, it can't provide it on its own. Models have been trained on data, and there's a cut-off date here, which means that they don't know things that happened after that particular cut of date. This is where function calling comes in. You can set up functions in your application that the model can call to retrieve specific data over form actions. For example, if your application need to check about the weather at a specific store location, which is real-time data, or stock levels, which is also live data, you can create a function that handles that request and returns the necessary data. The model can then integrate that data into its response, making the overall interaction much more dynamic and useful for the user. Now how does this work then? How does the model know about what functions we locally have? Well, let me explain using a graphical overview of what is really happening. It all starts with your application sending a request to the API. Along with the prompt, it will also include definitions of any functions that the model can call. So functions matter in C# really, which can be invoked back from the model. These functions need to have a good description of what they do. Once the model receives your prompt, it analyzes the input and decides whether it should respond directly or if one or more functions need to be called to provide the best response. The API then responds to your application, telling it which function should be called and providing the necessary arguments to pass to function. Next, your application executes the function with the arguments it received from the model. This might involve retrieving real-time data or performing an action based on the function's purpose. Finally, your application sends another request back to the API of the model, including the result of the function call and the original prompt. The model then processes this new information and generates a final response, incorporating the result from the function call. Function calling is something you'll often use. There are many cases that we can find here as use cases. A classroom example is getting real-time information like weather information or stock prices. In this case, when the user asks about the current weather of stock market updates, the model could call a function that fetches this data from an external API. Another example is making appointments. Imagine a user asking the model to schedule a meeting or an event to the calendar. Here the model could call a function that works with the user's agenda, passing the date, time, and details directly to the calendar system. Yet another good use case is finding the best price for a product. The model could search multiple online retailers or databases, comparing prices and availability, and then return the best option for the user. Adding functions for function calling is, well, pretty simple. Here is a function, and as we can see, it is just a plain C# function. To make it into a function that can be invoked on a model, we create a ChatTool instance using the CreateFunctionTool. That points to the function name and also contains a description I've mentioned, which is then passed to the model so that the model knows about our local function. Finally, we then integrate chat completion options, passing in the function or typical functions that you will

expose. These options are then passed in, and we'll see that in a demo.

Demo: Using Function Calling

Let's now explore this example here, which introduces a powerful feature, function calling. The scenario we will use revolves around a user asking about going to the nearest Bethany's Pie Shop and what clothes they should be wearing, which requires both a location and the weather data. That is data that the model doesn't have. So by default, we wouldn't be able to get an accurate response. But by adding function calling, we will. And the first part of this code introduces two custom chat tool objects. The chat tool can be seen as a means to complete a chat request. The model will indicate once it knows about the tool that it needs to be invoked, and its response will be used as an enrichment to the next prompt sent to the model. This first one here, getClosestBethanyPieShopStoreTool, helps the AI find the nearest Bethany's Pie Shop based on the user's current location. The actual method that will do this in C# code is the GetClosestBethanysPieShopStore, which we see here. This could in real life use geopositioning to locate the user. The description is also very important since this will be sent initially to the model so that the model knows what extras we can provide. This second one, getWeatherAtBethanysPieShopStoreTool, is, well, largely similar. It provides weather information at a given pie shop location. It also describes the parameters for the getWeatherAtBethanysPieShop store method since those will need to be provided by the model when it invokes this method. The first parameter is the location, so the city and the country, and the temperature unit to use. The second one is marked as optional. And note, it is also of type string. Remember, th e model will indicate that we will need to invoke this method in our application. So it's also the model that needs to supply our code with parameters. That is why it's important that we describe this correctly. Next, the main functionality is this method here. We're using again a regular ChatClient instance. Next, we have this user message, which isn't really asking about the weather, but it asks what clothes I should wear. The model will infer it needs to know what the weather is like. So it will invoke the tool automatically and then translate that response to knowing what to wear, as we'll see in just a second. To enable the model to perform dynamic tasks, so invoking the tools, we pass these two tools, so the closest store and weather tools into the ChatCompletionOptions. Next, we enter this while loop, which manages the conversation. This loop continues running until the AI finishes its response. In each iteration of the loop, the chatClient.CompleteChat method is called, which sends the user's message and the available tools to the AI. Based on the user's input, the AI determines whether it can answer the question directly or whether it needs to call one of the tools for additional information. The switch statement checks the responsible FinishReason to handle different scenarios. ChatFinishReason.ToolCalls is the most important case for us right now. It occurs when the model recognizes that it needs

more data to fulfill the user's request. In this case, the AI wants to call one or more of the tools we have defined. If the AI calls the GetClosestBethanysPieShopStore, we invoke the corresponding function to retrieve the location of the nearest Bethany's Pie Shop, which returns Brussels, Belgium. This information is then added to the conversation as a tool chat message. Next, if the model calls GetWeatherAtBethanysPieShopStore, it extracts the necessary parameters, that is the location and unit, from the request. In this case, the location is Brussels, and the temperature unit could be Celsius. The tool is called with these arguments, and it responds with 25 °C and raining. It is Belgium, after all. Again, this information is added back into the conversation for the model to use. That is, of course, crucial. Once the required tool calls have been completed, the requiresAction flag is set to true, which signals the loop to continue processing with the new information. The ChatFinishReason.Stop indicates that the AI has finished its response without needing any tool calls. In this case, the conversation is complete for this turn, and we simply add the AI's final message to the conversation history. The others are less relevant for us right now, so let's skip these. In case a tool call was done, we go back here, passing in the results to the model. And in normal situations, it will then get back a regular response, now with extra information from the tool calls. And we can then show that to the user. Let's see this in action. So using that extra information, the model could create a full response, answering what I should wear. And with that, we have definitely covered the most important functionalities of the OpenAI APIs by using it from .NET and using the OpenAI .NET API library. This was already a lot of fun if you ask me.

Adding OpenAI to an ASP.NET Core Blazor Application

Demo: Exploring the Finished Code

You've seen the console app. Now, see the real app. Sounds a bit like a promo for a movie, right? Well, although I'm not promoting a movie here, I'm promoting that we'll now learned together how we can add the functionalities that we've added using OpenAI in a console app, but now in a real-world ASP.NET Core Blazor app. You're typically not going to build console apps. Instead, I want to show you real-world scenarios where adding AI through OpenAI actually makes sense in a useful way. And we will start straightaway with the first demo. What we are looking at here is pretty much the finished application for this module. We're looking at the Bethany's Pie Shop Blazor application, specifically created for this course. And it mostly consists out of administration screens, and there's also a few public pages. I won't be covering the basics of Blazor. We have several other courses for that including Blazor Fundamentals where this app's foundations are

created. I will focus here on what is needed for our AI needs mostly. So I can log into the application. And then indeed, we'll see several administration screens in the menu. We'll start with this pie over here, and I'll show you the code in just a minute to show you also the general structure of the project and its architecture. It's nothing complex. Don't worry. So you have the pie overview, which shows all pies in the database, and we can also see the details. There's also the option to add a new pie, but we'll look at that in the next demos. Let's look at the code for the pie overview first. The PieOverview.razor is a very simple Blazor component that can only be accessed if you are logged in with an account in the administrator role. The most important part of this page is the foreach loop that will go over all pies in this Pies, showing each of them in a row in a table. At the end, we see the button that can navigate to the details. There's also a PieOverview.razor.cs, which is the component class with the code-behind, and there we can first see that list defined, so pies. Then, I have a PieDataService class, which is injected by using the Inject attribute, the typical way of injecting a service in a component in Blazor. That PieDataService instance is then used in the OnInitializedAsync to get all the pies from the data store and assign that list to the pies. The PieDataService is defined within interface IPieDataService, which we see here. It contains a definition of the most common things we'll do with pies in our application, so getting a list of all pies, getting the details, and adding, removing, and editing one. In the implementation through the actual PieDataService class, we can see that now an IPieRepository is injected, and that is used to invoke different methods on. The IPieRepository and, in turn, its implementation, the PieRepository, are using EF Core to get items from the database. I'm using here the DbContext, the ApplicationDbContext, which we get through the DbContextFactory, which is injected. In Blazor, it's preferred to use this approach instead of injecting directly the DbContext itself. In the GetAllPies, we use the context to get all the pies asynchronously. The GetPieById gets the pie based on the passed-in ID. And the add, update, and delete are pretty straightforward, as you can see here as well. All this goodness works because of dependency injection, which is configured in the Program.cs. Here, you can see that I'm registering several repositories and services, all inside of the Program class. Now while we're here in the Program.cs, I want to show you also that I've registered the OpenAIClient through dependency injection. Of course, the package has been added first, that we can see here in the project file. I've also created a secrets file again, which contains a ModelSettings entry containing the name of the models we'll use, as well as my API key. I have a ModelSettings class that contains the same properties by name. And in the Program.cs, I'm now going to use this. In this block, we're using a factory function to create the instance of OpenAIClient. The factory takes an argument, sp, which stands for service provider. The service provider is responsible for resolving dependencies, so getting instances of services already registered in the DI container. Next, we retrieve the ModelSettings object using sp.GetRequiredService, passing in IOptions of ModelSettings. This is where we are using configuration that has been injected into the application. Finally, we create and return a new instance of the

OpenAIClient class, passing in the OpenAI API key that was fetched from settings. This ensures that the OpenAIClient is initialized with the correct API key, and this cannot be injected, well, anywhere in the application. All right, now we are ready to AI-ify our application.

Demo: Creating the Add Pie Page

And now that OpenAI is configured and you have a high-level understanding of how the application is structured, it is time to start bringing in those AI features. And we'll start with looking at what features I have added to the Add Pie page. Adding a new pie or, in general, new content for a site, can be a difficult task. Not everyone is the next Shakespeare, and neither are the people at Bethany's Pie Shop that are managing the site. So let's look at the Add Pie page in action. Let's say that a new blueberry cheesecake has been added to the stock. That's the input we'll add here as the name of the pie. Then, we have the ability to enter a short description, but we can also generate one. We could, of course, go to ChatGPT, but it's much easier to stay in the context and generate one right from within the page. I can click on this Generate button, which will ask OpenAI to generate one based on the input and the prompt that sits behind this button. Look at how nice and easy this came into the screen. I've added the same behavior for the long description. Both actually have a length set for the content I want the model to generate. And instead of paying a photographer, why don't we have an image generated for us as well, again, directly inside of our application and so inside this page. That looks pretty nice. I can now set the price. That I can still do myself. And I can now click on Submit to add the new pie. And now the pie appears here in the overview. Let's see how all of this was achieved in code. The PieAdd component uses a regular Blazor EditForm with a couple of input text fields. Next, the component also contains the different buttons with event handlers in the component class. The model for the form is set to a pie instance, which will be used to capture the entered or generated input. In the class, I am now injecting the OpenAIClient since that is the one registered, scoped in the Program.cs. In the OnInitialized, which happens on initialization of the component, I use the GetChatClient and GetImageClient to get the client instances to communicate with the OpenAI models. The code inside the GenerateShortDescription should be pretty familiar since we have already covered this earlier. We created a well-crafted prompt that sets the scene, and I limit the length here. Not too many options this time, but as part of the prompt. It also works. Then, we use streaming again, and that worked nicely in the UI since Blazor will update the contents of the input continuously. Finally, I have the await foreach again that will get in basically any new piece of content, and I append that to the already existing value for the Pie.Description property. Now I do have to call the StateHasChanged method since Blazor otherwise doesn't perform an update of the UI. The long description generation is, of course, identical apart from the length, which is set here to

100. Finally, we have the generation of the image, and that code should be familiar too. First, we clear the current image in the UI, as well as the URL property. Then, again, I have a very descriptive prompt that explains in detail how the image should be generated. Then we bring in image generation options, those we have already covered. And then we call the GenerateImageAsync method, passing in the prompt and the options. That will hopefully, after a while, return us a set of bytes representing the image. I write that file to the disk. We could opt to put the file somewhere else, but that's not relevant here. And I also create an HTML string, GeneratedImage, which I show in the UI using the MarkupString, which will basically add the HTML string in the UI. The Pie ImageUrl is also updated so that when we add the pie by clicking Submit, all the data is ready for the service and the repository to save it to the database. So there we go. We have created an AI-enabled way to help the user in creating new content without leaving the actual environment. How easy was that?

Demo: Working with Tickets

Another area where I think we can use AI to enhance the experience of our support team here at Bethany's Pie Shop is support tickets. In the public site, I have created this page where the end user, so our customers, can create a support ticket. Let's first add a new ticket here. So we've entered here some text for the ticket. We're going to mark this as a complaint, and it is not really related to a certain pie. And I'll submit this ticket. If we now go to the admin tickets page, we can see a summary of the message and, eventually, other messages that were added to the support thread, giving us an easy way for our support staff to get an idea of the state of the message. When we open the ticket, there's a ticket information, and we can also see the score, that is a satisfaction score created by the model based on the user message. And since this was a negative-sounding message, it received, in this case, a 1. If we go back to the original ticket and we add an extra message here, saying that it actually was delicious so I already ate it. And we save that and we go back to the admin page, we can see that the summary has changed. So indeed, if we open the details, we see that the summary has changed, and also the score has been updated. So summarizing the ticket messages and creating the sentiment score is all done here using an AI model. Let's see how that was done. If we look at a public page to add a new ticket, we can see that I'm using a ticket with TicketMessages. On one ticket, we create a message, and every time the user adds to the thread, a new ticket message is added onto that ticket. I then have the TicketDataService, which is registered over here using dependency injection. And so an instance is injected into this component, for which I then call the AddTicket method, passing in the ticket instance, which contains the ticket messages. Let's look at the AddTicket method on the TicketDataService. And this does a few things, like changing some of the data. But most importantly, it calls SummarizeTicket method. Let's look at that one. In this method, we can see an

array of scores. I call them here satisfaction scores. They contain words expressing how a customer feels. Next, we can see the prompt, quite a long one, but it does quite a few things too. First, I'm explaining what the model should be doing. Then, I combine all the messages from one ticket since I want to create a summary of all messages. So that is done through this code here to loop over all the messages. Then, still as part of the prompt, I want the model to do a few things. First, I want to generate a 30-word max summary also indicating what should be done next. Then, I want to generate the title. And finally, I want to map to the satisfaction score array the latest message since that gives our support agent an ID or the state of mind our customer is in and how to react. I do that by first summarizing the last message, and then we map to one of the items in the array. In the end, I want the model to return me a JSON string of which I specify the structure here. What follows isn't code that is really related to AI. Of course, first, we'll invoke the model, but then we're going to deserialize the response and use that response to capture the value of the ticket summary and the title. And then I'll also try to map the score to one in the array of strings. All that data is added onto the ticket, which is then saved in the database, right here in AddTicket method. So this gives us another very useful way to bring AI into our application, adding summaries and satisfaction scores.

Demo: Creating the Sales Bot

We have seen function calling as a means to expand what a model can do. in this demo, I'm going to show you this sales bot inside Bethany's Pie Shop, which is going to help with getting insights into sales data. Here is the running page where we can ask questions like please compare the sales of January 2023 with January '24. Let's enter that here. Of course, OpenAI's models know a lot, but they probably have not been trained on the sales data of Bethany's Pie Shop. That's internal data, so it shouldn't be known by a public model. But looking at the response, it does seem that OpenAI knows about our sales, and it has generated some text based on that data even. That is possible again through function calling. Let's look at the code for that here. In the SalesBot.razor.cs, so in the component class, I've created this OnEnterQuestion event handler. This is going to be invoked when I enter my question. It might be that the bot can answer without needing to use an external tool. But chances are, it can't. So I'm going to create a function and wrap that in a tool that is made available to the model again. I have an OrderDataService that will return the number of pies sold for a given month and year. Remember, when we ran the application, we asked the model to compare sales data between 2 months of different years. So in the end, this code has been evoked. I'm then using a simple link array, adding the pies from the different orders and returning the sum. If there are no orders in that month, I'm returning -1. This function is then, let's say, exposed again through a tool. And it is the GetNumberOfPiesSoldInAGivenMonthAndYear tool which points to the corresponding method and comes with a detailed description. I'm

also including that -1 means that we cannot find any orders in the given month. And finally, I'm also describing the parameters that the model will need to add when using the tool. These are, again, the same parameters as we have in the actual method. Back in the OnEnterQuestion, I'm creating first the list of messages and then the ChatCompletionOptions, passing in the tools or, in fact, just one tool here. Next, I've again created a while loop that will look at the response coming back from the model after creating the CompleteChatAsync method. Inside the switch, which we already explored in great detail in the previous module, we will again look at what is the finish reason? If that is a tool call, it means that the local tool needs to be invoked. For simplicity's sake, I only have one. And if that's the one we need, which I hope, we parse the JSON that the model has sent along. We search for the month and year and will throw an exception if this cannot be found. Otherwise, the method is invoked locally, and the response is added to the list of messages. And that is then sent off to the model again. After that process has finished, probably multiple times since we asked to compare the sales results of different ones, we'll get a response, which is the one that we show in the UI. So there we go, a nice way to use function calling inside of our application.

Adding RAG

If we think back of what we have already seen about models, while extremely useful, they have their limitations. One of the key things to keep in mind is that these models are not updated in real time. They have a specific cut-off date for the information they were trained on. So while they can generate responses based on an enormous amount of data, they are limited to knowledge available up to that point in time. Another important limitation is that these models don't have access to internal company information. For example, if you have a company knowledge database, models won't have direct access to it unless you have explicitly integrated that information into the system. That is where function calling comes in. By integrating external APIs, databases, or even specific tools, function calling allows you to extend the capabilities of the model. You can pull in real-time data or allow the model to interact with systems it wouldn't normally have access to. Now there is another technique that we can use to create better output from a model, and that is RAG, short for retrieval augmented generation. RAG is a powerful approach that enhances the outputs of AI models by pulling in relevant external information before generating a response. So instead of solely relying on the model's built-in knowledge. which like I just said can be limited or outdated, RAG allows the model to fetch and use fresh, relevant data before responding. This ensures that the information that is generated is more up to date and accurate. So how does a retrieval augmented generation actually work? There are two key steps. First, retrieval. Here, the model fetches relevant data from external sources. Then there is generation. The AI model uses that retrieved information to enrich its response. This combination

of retrieval and generation helps the model to provide better context-aware answers. It is no longer just relying on internal knowledge. No. Instead, answers are grounded in real-time data, which means fewer mistakes or hallucinations. Now you may be thinking, where can I use this then? Well, let me give you some examples. One example could be customer support. When customers ask questions, RAG can help retrieve up-to-date information from knowledge bases or FAQs, providing accurate and quick responses. Next in legal assistance. Staying up to date with laws and cases is crucial. RAG can pull in recent legal references in case studies, allowing professionals to give more informed advice. Finally, in health care where accuracy is a matter of life and death, RAG can retrieve the latest research or patient data, enabling better decision-making and more personalized care.

Demo: Adding the Pie Creator

Now function calling, which we have covered for the sales bot, is a way to make information external to the model available for the model to consume. We've seen the back-and-forth process between the model and our application. However, we don't always need to use this approach. And instead, we can use RAG. Take a look at this page, the Pie Creator, on which I have applied this principle. Say that we have some ingredients still lying around, and we want to look in the list of pie recipes which pie we can still create with that. I'll paste in some ingredients, and we'll ask the model to figure out which pie or pies can still be created. It found correctly that a pumpkin pie can still be created. Now I'll change this to one egg. It, again, correctly says that it cannot find a pie that can still be created using the available ingredients. How did it do that? Well, I have applied a basic form of RAG here. Let me show you. In the database, I have through data seeding added a large list of pie recipes, including their ingredients. What I want is that the model will search in these ingredients for a possible match. If you think about it, that is not what we were doing with function calling. Here, we are basically going to feed the model the entire list of all recipes as part of the request that was sent. We are retrieving that data first, and then that data can be used by the model. It is augmented with that data. So indeed, in the Pie Creator, I first retrieve all the data through a data service, and that will be returned as JSON. So to be clear, this retrieves all recipes from the database in a JSON format. In the prompt, I am explaining to the OpenAI model again what it should do, that it should search for a pie that can be created with the passed-in ingredients. But, and this is the augment part, I'm going to include the entire database of recipes as part of the prompt. And from then on, it's on the model to figure out if it can find a recipe that can still be created with the passed-in ingredients. This is a simple but effective way to create a RAG-based solution.

Exploring the Different Azure AI Services

for .NET Applications

Introducing Azure AI Services

When we think about AI, we'll quickly think about OpenAI and GPT. But OpenAI isn't the only player in the AI field. Far from it. Microsoft has a whole set of AI services, part of Azure called Azure AI services that you can leverage in your .NET apps to do even more. And it even existed before CahtGPT became as popular as it is today. Let us jump straight in. So let's start with understanding what exactly Azure AI services is then. Well, it is a set of cloud and AI-driven services, which makes it simple to integrate some pretty advanced artificial intelligence into our applications. It is a pretty extended set of services that brings, well, quite a wide variety of services, including understanding natural language, recognizing objects and images, or converting speech to text. And yes, so much more. You can think of them as prebuilt models accompanied by APIs that handle a lot of the complexities behind the scenes, again with the goal that we don't have to worry about the nitty details of machine learning. We can focus on what we normally build. .NET apps and Azure and Azure AI services will do the heavy lifting. Azure AI services are fully cloud-based. As said, these services come with pretrained models that have been built using vast datasets, and so we can be productive pretty quickly. Just like what we have seen with OpenAI, to use it from our apps, we don't need to know machine learning or be an AI expert. Far from it. These tools are aimed at lowering the bar to integrate AI into our apps. If needed, you can also customize the services to better suit specific use cases. And the fact that these models are accessible via APIs and, again, also using SDKs makes integration into your .NET applications straightforward. SDKs exist for C#, which is what we are after, but they also exist for other languages and platforms. So what services are offered by Azure AI services then? Well, to start, it's a large set, and it is by no means my goal to show you each and every service here in this course. After all, our goal is showing how we can integrate them into our .NET apps. The different services are grouped into several categories, such as language, translator, speech, vision, search, and documents. Each category comes with services designed to solve different types of problems. For instance, the Language service can help analyze text for sentiment or key phrases, while the Vision service helps analyzing and tagging images. As said, I'm not going to show you each service, unless you have a few hours to spare? No. Well, then I'll select a few of my favorites and show you in this module how we can use them from .NET. Before we continue, a small side step. Next to Azure's AI services, we have the Azure OpenAI service, which gives us access to the models developed by OpenAI such as GPT. This service allows you to integrate powerful language models directly into your applications by deploying your own instances with the security and scalability which use from Azure. So the focus of the Azure AI services is on general AI capabilities like speech or vision;

whereas Azure OpenAI service brings you the capability to deploy and use your own secure and private instances of the AI models. We won't be covering Azure OpenAI service in this course as working with it is similar to working with OpenAI directly, although more options are available. So, let's now start to understand more of one of the most commonly used Azure AI services, and that is Azure AI Language. Azure AI Language service is all about understanding and processing text. Whether you need to analyze user feedback, detect sentiment in social media posts, or recognize specific entities within a document, this service can do it. It comes with many features such as, already mentioned, language detection, text analytics for doing things like sentiment analysis, summarization, and much more. It also supports custom models, meaning you can train the service on your own data to get even more precise results.

Demo: Setting Up Azure AI Language

All Azure AI services, and there's also Azure AI Language, are at the core Azure resources. So we'll need to set that up first in the Azure portal. Once we have created and deployed the resource, we can start consuming it. Now for that, we'll again need an API, which I'll show you we can find in the portal. That key can then be used from our apps to make subsequent requests to our Azure AI Language endpoint, and I'll show you all that in this demo. So I am here in the portal, and I have created a resource group already in which I'm going to place all the AI services that we are going to work with in this module. And so as said, we will start with the creation of the service. So I'll click here on Create, and then we'll search for language. And we'll find the service we are looking for. Here, I will select that, indeed, I want to create the language service. And if we want, we can also add additional features. That won't be needed for the things we are doing here, so we'll select to Continue here. In the next screen, let me quickly add the necessary details here. I have selected the resource group we had earlier. I have selected a region, given the resource a name, and selected a pricing tier. Typically, you can select the free tier here as well if you're not using that one already in your subscription. I have already used it as you can see, so I'll need to select a paid option here. Also, note that not all services are available in all regions. Finally, we will check this box here, indicating that we will use this service responsibly. And then we'll click on Review + create. After a while, the resource will have been created. Here is the service now in the resource group, and we can now go and grab the API key and other information that we need in our application. Under Resource Management, click here on Keys and Endpoint. And in there, we'll find the information we need. We'll, in fact, need two things in our code for the communication to work, and it is this key and this endpoint. With these, we can now head over to Visual Studio. I have the finished application for this module open here, and I'm inside the launchSettings.json file again. Since this is a console application once again that we'll use to learn about the principles of Azure AI services, there's, again, very little to secure here. So I'm going to store

my secrets directly in this file, like we did earlier. Under the environmentVariables, I have now created a couple of key value pairs that I'll use from code. This is the language key, which contains this copied value, and this is the language endpoint, also copied from the portal. With these values in the application already, we are ready to start communicating with the services from C#. Let's learn more about that now.

Using Azure AI Language

When it comes to working with Azure AI services from our apps, there are, again, basically two approaches we can take. First with the HTTP API, you're sending raw HTTP requests directly from your .NET code using classes like the HttpClient. This gives you a lot of control but may require more effort to handle things like retries or errors. Azure AI services indeed expose their functionality again through a REST API, just like what we've seen with OpenAI. The SDK, on the other hand, wraps these operations again, which will make your life easier. And the SDK is available for most services. The SDKs provided by Azure make it really easy to integrate AI functionality into your applications, much easier than creating HTTP requests manually. With very little code, you can call any of the Azure AI services without having to worry about things like authentication details or retries. The SDKs are also well-maintained and are regularly updated. So any changes made to the services can be consumed quickly from code using the SDKs as well. And installing the SDK is as easy as adding one or more NuGet packages. And from there, you're ready to start working with Azure AI services. How can we now use the Azure AI services from code? Well, we have seen how to create an API key and where we can find the endpoint URL. I'm storing those again in environment variables. Then, we'll use the AzureKeyCredential class to securely manage the key. Next, we'll instantiate a TextAnalyticsClient class to handle the communication. So that is in the case that you want to work with Azure AI Language. That's a class that is part of the SDK, of course. Once that is in place, your application is ready to send text data to Azure for analysis. As mentioned, the SDK exposes the different functionalities which are available. For example, the DetectLanguage method can be used to automatically identify the language of a given piece of text. The AnalyzeSentiment method allows us to check whether the sentiment in a piece of content is positive, negative. or neutral. This can be useful to get an idea of how people are reacting on social media on your products, for example. The ExtractiveSummarize method is used to extract key points from a document or article giving you a summary. And the RecognizeEntities method can be used to identify and categorize different entities mentioned in a piece of text, such as names, dates, organizations, or locations and these are just a few methods. There will be many more. But again, it is not my goal to start listing out all of these here. Here, we can see the TextAnalyticsClient in action. After creating an instance of the client, we can use the DetectLanguage method to

automatically identify the language of a given piece of text. The service will return a structured result that is the detectedLanguage engine instance we can see here. And that includes the language code and a confidence score. The latter, so the score, lets us know how certain it is about its identification. Talking about language detection, it is definitely one of the key points of Azure AI Language. Imagine that you have an application that processes text like support tickets for multiple regions and languages. With this service, you can automatically detect the language of each input. We'll get back, as said, the detected language and a confidence score, which means that we can follow a different flow based on the language of the incoming text.

Demo: Using Azure AI Language

Now to see all the goodness offered by Azure AI Language in action in a demo. First and perhaps foremost, a package is required to be added, the SDK that is, in the project file. This one here, the Azure.AI.TextAnalytics package, is the one you'll need to bring in to perform all the actions we'll look at in this demo. Now, inside the project for this demo, you will find different folders, each containing examples for the different services we're going to look at in this module. In this demo, we'll look at this folder right here, the Language folder, which will contain quite logically the code for the Language service integrations. And let's start with looking at this file here, the LanguageDetector. We start by retrieving the API key and the endpoint URL from the environment variables using the Environment.GetEnvironmentVariable like we've done before. Next, I'm creating an AzureKeyCredential object using the API key. This object is passed into the TextAnalytics client, which we then initialize with both the endpoint URI and the key credential. This client acts as the main gateway for sending requests to the Azure AI Language service. And it's, of course, part of the SDK. So now that our client is set up, we move on to the core part, and that is language detection. We have a sample text description text, which is written in French. If you don't understand what it says here, it is basically a description of a cheesecake but then done in French. Fun fact, I generated this using ChatGPT because my French isn't as good as that from ChatGPT. Using the textAnalyticsClient, we call the DetectLanguage method and pass in our sample text. This will send the text to Azure's back end where the AI model processes it and will get back a DetectedLanguage object from the SDK. If we run this, we'll indeed see that the text is detected to be French. Perfect. And we could see how easy this all was using the SDK, right? Let us look at sentiment analysis next. We'll use again the TextAnalyticsClient class. And this time, I want to use the Azure AI Language service to get the sentiment of this piece of feedback here. I'm passing this feedback to the AnalyzeSentiment method, which performs, well, sentiment analysis on the entire document and provides an overview of the emotional tone. The response stored in this document sentiment includes the general sentiment of the text like positive, negative, neutral, and also the confidence

scores for each sentiment, which indicates how confident the model is about, well, its assessment. Those I'm displaying here in this code. Next, we iterate over the documentSentiment.sentences to analyze each sentence individually. For each sentence, we print the text of the sentence, the sentiment of that sentence, and again, the confidence scores for positive, negative, and neutral assessments. Now, you may wonder, why is this useful, right? Well, we can use this to pinpoint which parts of the feedback contribute to the overall sentiment. For example, sentences describing issues like the texture was grainy, not creamy, will likely be marked as negative, while sentences like the delivery was on time could be tagged as positive or neutral. Now if you want, we can drill even deeper, but I'll skip that here. If we run the code here, we see the following. The text sentiment is mixed according to Azure AI Language. Indeed, I see 16% positive and 83% negative. Pretty mixed, if you ask me. Then it continues showing the sentiment for each sentence separately, and that is what we see here. Let's look at two more options of Azure AI Language. And let's first look at the option to recognize entities in content, which is a powerful way to extract meaningful elements from text and categorize them. This can help gaining insights from documents and content. I'm using as description again a text that describes in much detail a cheesecake. And then, I call the RecognizeEntities method on the textAnalyticsClient. If the extracted entities collection has any results, we iterate through each entity and print the details, including the text, the length, the category, the subcategory, if available, and finally, the confidence score. Again, in this cheesecake description, entities like cheesecake, vanilla, and sour cream could be identified and categorized, giving a clear breakdown of the key elements mentioned in the text. So, let's run this to verify. And yes, we indeed get back the entities as hoped, including the cheesecake, the cream cheese, and quite a few more. They have also a category applied onto them. They are products, and there's one here at the bottom, presentation, which is categorized as a skill. Now I think this is quite cool. Let's do one more thing, and that is summarizing text, again using the TextAnalyticsClient. This time around, we will try to extract the main points from a long piece of text using an extractive summarization approach, allowing you to distill content into a concise summary. To summarize the content, we put story into a list of string called documents. Then I call the ExtractiveSummarize on the TextAnalyticsClient and pass in the WaitUntil.Completed to ensure that the operation runs to completion. The result is an ExtractiveSummarize operation, which contains the summarized content once the process finishes, I use an await foreach loop, which iterates over each summary page. And within each page, we further iterate through the summaries and print the result. And for each extracted summary, we then display the number of sentences in the summary, showing how concise the output is. And then I print each sentence individually, providing the main points distilled from the original content. Time to see this one in action to write. Well, there we go. The summarizer ended up with one summary page containing three sentences as we see here. Now these demos should already give you a good feel of the way we can work with Azure AI Language from code. And yes, there are

definitely more options, but they're all quite similar. So I suggest you explore these more on your own if you want.

Demo: Working with Azure AI Vision

Let us shift gears now and talk about Azure AI Vision. This service provides powerful capabilities for analyzing images and videos. The features of the service include detecting objects in an image, performing facial recognition, extracting text from images using OCR, or even performing video analysis. Now, seeing a demo where we can recognize an object in an image is cool. But what about where we use this for real? There are absolutely many real-world use cases for Azure AI Vision. For instance, you can use it to moderate images by automatically flagging inappropriate content. Or if you're building a media library, Azure Vision can help you, for example, to tag and categorize images based on their content. It can even generate automatic captions for images, helping with accessibility. Time for another demo where we will see how I can use the different functionalities of Azure AI Vision. I've again prepared a few samples, which use Azure AI Vision. These demos are using the SDK, which is referenced here in the project file. Let's now look at how we can have Azure AI Vision describe an image for us with the snippet we see here in the ImageCaptionGenerator.cs file. This time, we're using an image analysis guide, and I'm going to use this image here, the Store.png, which I, in fact, already created using DALL-E. We call the Analyze method on the ImageAnalysisClient, passing in the image data as a binary data object and specifying VisualFeatures.Caption as the visual feature to analyze. This tells the service to generate a description or caption for the image. The result we get back is an ImageAnalysisResult, which contains the generated caption and, again, a confidence score that indicates how certain the model is about its description. Let's run this and see the result. Well, that looks quite okay to me. However, it seems that Azure isn't so sure as we can see that the confidence score is pretty low. But what we see in any case is that using the service, it is easy to extract meaningful insights from visual content. This sort of capability can be integrated into various applications. And we'll use this also in the next module when we integrate this into Bethany's Pie Shop to generate alt texts for images in our site. In a very similar way, we can also generate tags using the ImageAnalysisClient, and that's what I'm doing here. This time, I do specify that I want to generate tags instead of a caption. The result is, again, stored in an image analysis result object, which contains an array of tags. Each tag represents an attribute or element identified in the image, again, along with the confidence score, indicating how certain the model is about the accuracy of that tag. Let's run this, and we'll get a number of tags. We see that it is a scene, it's indoor, it has tables, books, and quite a number of extra tags. The extraction is particularly useful for a number of things. Think of content categorization where tagging images helps in organizing and managing image libraries, making it easier to search and filter content. Doing this manually is a time-intensive task that can

now be entirely automated. Let's look at one more option here, object recognition. Using this, we can identify and localize specific objects, complete with bounding boxes that outline the position of the elements in the image. Again, we're using the Analyze method. But now I've specified that I want to identify objects using this VisualFeatures.Objects parameter. We're again getting back an ImageAnalysisResult, which contains a list of detected objects. And for each, we'll get a tag, so what the object is, as well as the bounding box. Let's see this in an action. There we go. It identified some items in the image, well, only chairs in this case. And for each chair, it has also indicated where it found them too.

Demo: Using Speech Service

Let's now take a look at Azure Speech service, which offers functionality for converting speech to text, text to speech, speaker recognition, and even translating spoken language in real time. This is especially useful for applications that require voice control, real-time transcription of meetings, or multi language communication. Azure Speech allows us to include voice commands in our application or automatically transcribing audio recordings into text. And yes, again, I am just scratching the surface here. This service tool can do a lot more than what I am discussing here. Let us see the Azure AI Speech service in action. As before, the SDK package is referenced here in the project file again, and it is this one in this case. For the files that we'll look at, just two by the way, we need to head over to the Speech folder. Let's first look at working with an audio file, this one here, which I have included in the project. Note that it's also set to copy to output directory always so that the executable can access it. Okay, back in the SpeechFromFile, we're using this time the SpeechConfig to configure the connection with Azure. Note that this also expects an API key in a region, not an endpoint, which is different from the ones we've already used. I won't show you how to set it up in Azure to save us some time. It's all very similar. Next, the SpeechConfig object is set to en-US as the speech recognition language, which means that the service will transcribe English audio specifically. This configuration ensures that the recognizer processes the audio input with the appropriate model. And to perform speech recognition, we need an audio source, and the code uses the AudioConfig FromWavFileInput, passing in gill.wav to create an AudioConfig object that reads audio data from a wav file. The using statement here ensures that the audio resource is properly disposed after use. A speechRecognizer, that's important for us here, is then created using the speechConfig and the audioSource as parameters. This recognizer is responsible for processing the audio and transcribing it into text. The RecognizeOnceAsync method is called to perform the actual speech recognition, and the result is returned as a RecognitionResult. So the result is stored. And on that, we use the Text property to see the transcribed text. Pretty neat, isn't it? Let's see if this actually does work. The audio file that I provided is a piece of content from another Pluralsight course of mine. So it should transcribe that. Well,

hopefully. And yes, that is sort of the language that was in that file. It didn't understand everything 100% correctly. Now instead of using an audio file, we can use real-time speech, so coming from the microphone. Most of the code is the same as we see here. I'll also use a speechRecognizer again. And on that, I'm again using the RecognizeOnceAsync method. Then I have this switch statement here to handle the different results of the speech recognition. If the recognition is successful, the recognized text is displayed. If no recognizable speech is detected in the input, we simply display this message. If the operation is canceled, details are extracted using the cancellation details FromResult, passing in that recognitionResult. And also, we'll print a message for the reason of cancellation. Let's run this now, and let me speak into the microphone. To be or not to be, that's the question. And there we go. It recognized the text correctly.

Demo: Using Text Translation

In the very final demo of this module, let's look at yet another service, text translation. I think you can pretty easily guess what this service will be doing. For this code to work, I've added again a package, this one here in the project file. The code can be found in this TextTranslator.cs file. Here I, again, need the API and the region to create a TextTranslationClient, the class part of the SDK that we use here. The target language for the translation is specified as nl, which stands for Dutch. Of course, I also pass in the text that needs to be translated. Then I'm using the TranslateAsync method, passing in the target language and the source text. This method, quite logically, sends a request to Azure's Translation service. The value property of the translation response holds the list of translations. And the method selects the first item from this collection as the translationResult, which contains details about the detected original language and the translated text, which I'm using here. And I'm finally also displaying the translated text in Dutch. Let's run this. There we go. As a native Dutch speaker, I can testify that this text is indeed in Dutch and looks quite okay.

Integrating Azure AI Services in an ASP.NET Core Blazor Application

Demo: Generating ALT Text

We have seen how to use Azure AI services, but again, only from a console application. And still that won't be what we offer in our building. So next, we will do what we have done with OpenAI as well. We'll integrate Azure AI services in our Blazor application for Bethany's Pie Shop. So we'll dive straight into our Visual Studio environment again, and we'll see how we can add Azure AI services and actually use them in a practical way. So in this demo, we'll do a couple of

things. We will first start with the configuration. So I'll bring in the correct packages, and I'll perform some changes in the Program.cs, some required and some to make our lives a bit easier. And then, we'll go and generate all text from an image. When we looked at the console application, we used the Azure SDKs to connect with the Azure AI services instead of just making plain HTTP calls, which would, to be clear, also be possible. We are, of course, going to do the same thing here. I'm going to bring in the packages again in the project file as we can see here. I have brought in a few, covering the different SDKs we'll use in this module. For now, this one, the Azure.AI.Vision.ImageAnalysis, is the important one since we will use the ImageAnalysisClient again soon. Now this is a Blazor application, and thus we have the Program.cs that does a few things, including registering services, so classes or types basically, with the dependency injection system of ASP.NET Core. In these lines here, I'm going to do something similar to what we have done before with the OpenAIClient, and that is registering the ImageAnalysisClient with the Services collection so that we can use it anywhere in the application. And no other changes are needed in the Program.cs. So, let's head over to the Pie Add, page which I have now extended. In the Razor code, I've added another input text where the user can enter the alt text for the image, which is again simply using an input text, which becomes a regular HTML input when rendered. But since this is a course about AI, we're again going to sprinkle some AI on top of that. So there's again a button that will trigger the execution of code. It will trigger the GenerateAltText method. That contains code that will indeed generate the alt text for the image we have generated and saved in the steps before. So the input is indeed the image that was generated by OpenAI, which we saved locally, and so we're using that public URL. We're using the ImageAnalysisClient, which is injected into this Blazor component through this code here. That works because we have just registered this type with the services collection. And on that, we call the Analyze method, passing in indeed the ImageUrl. Secondly, I'm passing VisualFeatures.Caption and VisualFeatures.Read. This specifies which features of the image we want to analyze. The caption feature tells the AI to generate a brief description of the image's content, while read may instruct it to analyze text within the image if there is any. Combining these two options allows the method to capture both the main content of the image and perhaps any embedded text. Finally, I then save the AltText coming in from the Caption property here. So, time to run this. One thing to keep in mind is that the ImageUrl will be passed to Azure AI services and thus needs to be publicly available. If you're running this from your local machine, your image will have a localhost address, and that won't work. So here is the running page with the other content already generated. I can now click to generate the alt text, and that's now coming in nicely by having Azure Vision look at the image and describe it for us. This text can be in the public part of the site to increase the accessibility of the page.

Demo: Adding Sentiment Analysis

In this next demo, we're going to continue using Azure AI services more specifically and now bring in support for sentiment analysis to analyze text entered by the user. I want to capture and store, with the help of AI that is, what the sentiment is of entered text. I have things set up in pretty much the same way as we saw with the alt text. First, we're going to analyze the sentiment of text. We're in the realm of the TextAnalyticsClient. So therefore, in the project file, I have now brought in a reference to Azure.AI.TextAnalytics again, a package for the SDK we also used in the previous module. Then, again, in a similar way, inside the Program.cs, I have now registered with the services collection the just-mentioned TextAnalyticsClient, passing in this time different parameters though. For text, we need to use the API key and the language endpoint. Both values we can retrieve via the Azure portal. Before we look at the actual code, let's look at the running sample so it's easier to understand what we'll build. In the application, basically in the so-called public part, a user can create a new ticket. We've seen that before. Let me create a new ticket quickly. There's a message, and that seems to be a complaint, right? So let me submit that. So let's now head over to the admin pages where we can also see the ticket and its details. On this details page, we can now see a sad smiley that appears because when the ticket was created by the user, in the background, I also went to Azure AI services and asked it to analyze the sentiment of the Android message. This is extremely useful because, at a glance, the support agent knows that this message is a bad one. So they should perhaps treat it with priority. Let's look at the code for this. Upon saving the ticket, so when clicking in the public page, this AddTicket is called on the TicketDataService. In here, we can now see that I've added this call to GetTicketMessageSentiment. In the implementation, I'm using the TextAnalyticsClient again. An instance, so this one right here is passed in through dependency injection again. And since we're not in a component here, I can use regular constructor injection. This time, since we're also in an asynchronous context, I'm also going to use the async version, so AnalyzeSentimentAsync, passing in the ticket message and an instance of options. I'm not using anything in particular here, but I could do so. The AnalyzeSentimentAsync method, as expected, processes the text, identifies the sentiment of positive, negative, or neutral, and returns a response in DocumentSentiment, which contains the sentiment analysis result. The result sentiment provides confidence scores indicating the likelihood of each sentiment category. Once we have the scores, we still need to figure out what is the overall sentiment. And that's what I'm doing here using this if, else if, else statement. The value is assigned to the TicketMessages.TicketMessageSentiment property, which is then stored in the database. In the admin ticket detail page, it is this value that is used to display the smiling on the page. I've added a couple of images here, and you can see that the image name contains the value of the sentiment. So there we go. We're using another service of Azure AI services in a useful way.

Demo: Detecting the Language

In this final demo of this module, we're going to add one more useful feature based on Azure services, and it is language detection. It'll probably start being a bit boring, so let's go over this very quickly. I've added another package for translations to work, namely the Azure.AI.Translation.Text package. And I've registered another type, this time the TextTranslationClient class with the services collection. Nothing different from what we already saw. What I have added for this sample is the option to allow the user to enter a ticket message in the language of their choice. So say that I'm speaking Dutch and I just entered the ticket message here in Dutch, not bothered by the fact that this entire site is in English. Upon saving, what I'm doing is checking the language of the entered message using Azure's AI ability to detect the language. If the language isn't English, I'm also going to store both the original text and the English version, which is created by Azure AI again. So here are the ticket details, we can see that this time it was seen that the message was in Dutch. We see the original message and the translated one. Now how is this done? Let's take a look. We'll have to look again at the TicketDataService and again at the AddTicket method. I'm calling here the GetTicketLanguage method, passing in again the ticket message, which could now be entered in a language different than English. First here, I'm going to check if the language is English. I pass the ticketMessage to the TextAnalyticsClient, and we use the DetectLanguage method again. The detected language is stored in an ISO language code, like en for English and fr for French. I store that value here on the ticketMessage instance. If the detected language is effectively in English, so the ISO code is en, no further action is needed. If not, well, then we go once more to Azure AI. But this time, we're going to use the TextTranslationClient in order to indeed translate the message. I call the TranslateAsync, passing in the target language and the message. The translation response, so this response, contains a list of translated text item objects. The method extracts the first item from this list, if available that is. And assigns the translated text to the ticketMessages.MessageEn property. This property now holds the English translation of the message, making it accessible to customer support agents who may not speak the original language. This value we are showing in the UI. Here you see the UI code for the ticket detail page. We always show the original message. And then if the language isn't English, then show the translated text as well. So there we go, another useful way to integrate Azure AI services into our Blazor application.

Understanding Semantic Kernel

Introducing Semantic Kernel

What if I say to you that there is a framework that not only enables you to work with different AI models but also makes that process simpler by abstracting the details away from us, as developers. That is exactly what Semantic Kernel, a

powerful AI framework for Microsoft, is created to do. In this module, I will introduce you to the basics of the Semantic Kernel library. Let's have it. So, what is Semantic Kernel then really? At its core, Semantic Kernel is a powerful .NET library created by Microsoft. And it's, of course, available as an open source project on GitHub. Its main goal is to make it easier for us, as developers, to work with different AI models in the same way, and it does this by abstracting a lot of the complexity that typically comes with integrating different AI services. Think of it as your AI assistant for building assistants, except you don't need to be an AI expert to get started. You don't even need to know about the API for the model. With Semantic Kernel, we can work with models like OpenAI or other models like Gemini while focusing on building applications rather than understanding the actual APIs. So you can now focus on the business logic and functionality, knowing that the kernel has your back, managing all the behind-the-scenes AI interactions for us. Semantic Kernel basically acts as kind of a middleware that makes it easy to use different AI models in our applications. The library is available for different languages, including C#, but also Java and Python. And from these different languages, we can use the same functionalities. Nearly all features are available across the board. Not only does Semantic Kernel streamline model interactions, it also standardizes them. So what I mean is that for a certain task, say text generation, you don't need to understand how a specific model does this via its API. You can use Semantic Kernel, and it'll figure out how to do this based on the model we're targeting. That makes things a lot easier, and it supports a wide range of tasks, including text generation, image generation, chat, and much more. There are several reasons to choose Semantic Kernel when integrating AI capabilities into your .NET applications. First, as said, there's abstraction. That's a huge advantage. The Semantic Kernel library abstracts away the complexities of interacting with multiple AI services. This abstraction makes it easier to switch between services or use them in combination without needing to manage each API individually. Secondly, Semantic Kernel provides seamless integration of AI into your projects. Where you need text completion, image generation, or chatbots, the kernel allows you to connect these services with minimal effort. And thirdly, another reason to use Semantic Kernel is its extensibility. Developers can easily extend the kernel to support new AI services or custom functionalities by adding plugins or external functions. This makes it flexible enough to adapt to changing requirements or incorporate new technologies when they emerge.

Using Semantic Kernel

So I think we're now all ready to see Semantic Kernel in action, right? So first, as expected, we'll need to add the Semantic Kernel NuGet package. Then, we need to configure the kernel to work with your preferred AI model. And, as we have already done with other integrations, we'll need an API key from the service that we're using. Pretty simple, isn't it? If you look at the name, it seems that kernel in

Semantic Kernel is pretty important. Indeed, it is the brain, the central component that coordinates and orchestrates all interactions between your application and the various AI models that we use. It simplifies the process of making requests to AI services by handling the underlying complexity. When you use the kernel, as said, you don't need to directly interact with the raw APIs of each service. Instead, you work using the kernel, which handles tasks like sending requests and handling responses. It supports services and plugins, and I'll talk more about those in the next slide. And from a .NET perspective, it acts as a dependency injection container, making it easy to add, modify, and remove AI services as our project evolves. As said, we can add different types of services to the kernel, typically AI services, such as text completion. And next, we can also use plugins, a very crucial feature of Semantic Kernel. These act as tools that extend the capabilities of LLMs, connecting traditional code, so C# code, with AI functionalities. With plugins, you can easily integrate real-time data, enterprise knowledge, or perform actions like sending emails or retrieving information from APIs. We'll talk more about plugins later in this module. So let's now look at some code with Semantic Kernel. First, we need to set up the kernel, which is what I'm doing here using Kernel.CreateBuilder. Then, I'm adding an AI service, in this case OpenAIChatCompletion, passing in the name of the model and the API key again. Once these parameters are set, I call the Build to create the kernel. This is essentially the foundation of our Semantic Kernel setup, which goes in the Program.cs. And yes, typically you'll add more services here too. With the kernel available, you will be amazed to see how easy it is to create a simple chat application. With the power of the kernel, interactions with the language model are just one or two function calls, making the process really efficient. The await kernel.InvokePromptAsync line is where the interaction with the LLM is happening. It takes the user's input using the Console.WriteLine, sends it to the LLM, and receives a string response back. Yes, it is that simple.

Demo: Creating Chat Completion with Semantic Kernel

Can it all be so simple as this? Let us verify in this first demo. I'm going to start with setting up Semantic Kernel in, again, a console application. And then, we'll do the most basic task we typically do with an AI model, namely chat completion. So as before, we will start with understanding things in a console app so that we can focus on what matters most, which is Semantic Kernel. And then later on, we'll go to Bethany's Pie Shop again and see how we can enable Semantic Kernel in a real application. But so first, I have again included an application that contains the code we'll work with in this and the following demos for this module. But before we look at the code, first, I have brought in the latest version of the Semantic Kernel NuGet package at the time of the recording of this course. The code for this specific demo can be found in the BasicsOfSK.cs file. We have a number of methods again, each highlighting a specific functionality. In this first one here, the SimplestPromptLoop, we are going to

create a very simple chat application In fact, I should say text completion application. The application contains, again, an API key in the launchSettings.json. So we'll use that to let Semantic Kernel talk with the model. As mentioned, Semantic Kernel is an abstraction. So it's pretty easy to switch between models without a lot, if any, code changes. So first, I begin by initializing a Kernel object, which really acts as the core interface, let's say, to the OpenAI model to Semantic Kernel. The code calls the Kernel.CreateBuilder to start building a new Semantic Kernel instance. Then, using the AddOpenAIChatCompletion, it configures the kernel to use OpenAI's chat completion model. Other methods exist here too so that Semantic could be configured to use Azure OpenAI, for example. Similar to what we saw with OpenAI, this method expects the model name and the API key. So by including these settings, the kernel has all it needs to connect to OpenAI and handle prompts. Then the Build function here finalizes and creates the kernel instance, making it ready to handle inputs and create AI responses. That is, essentially you can see that this is very similar to setting up the DI container in the .NET applications. And in essence, as said in the slides, we can think of the kernel as a dependency injection container in which we register all services, in this case only OpenAI chat completion. Then, with the kernel ready, we call the InvokePromptAsync method, passing in the entered text as the prompt for the model. As this be configured with OpenAI, it's essentially going to do the exact same thing as we did before, go and talk with the OpenAI chat completions endpoint and get back a response. And note that this method, this InvokePromptAsync, does quite a bit without us really noticing. It communicates with the model, submitting the prompt, processes it, and waits for the model to return. Okay, let's run this. But don't expect any miracles here. We can ask a question, and we'll get back a response, as we can see here. And we can ask it another one, and we'll get back another response. Although what happens is similar, I hope you see the power already. It's an abstraction that is talking with the model in a very simple and clean way.

Using Chat History

What we noticed in the demo is not new. We've seen this with other interactions with AI models too. LLMs don't have memory between interactions. They're stateless. To create more engaging and context-aware conversations, you need to maintain a chat history on the application side, so on our end. This chat history gets included with each chat request, giving the AI model the context it needs to respond meaningfully. In Semantic Kernel, maintaining the chat history is straightforward and easy to bring in. Just like with everything we've seen already. In this snippet here, we are creating a new chat history instance, a type that is coming from Semantic Kernel itself. This allows us to handle the chat history from our application's end. We start by adding a system message using AddSystemMessage. This message sets the context for the conversation, telling

the LLM that it is acting as a customer assistant at Bethany's Pie Shop. Next, you add a user message using the AddUserMessage method, simulating a customer asking for information about their order. This helps the assistant understand what the user is asking. Finally, we add an assistant method using the AddAssistantMessage method. The assistant asks for more details from the user, specifically requesting the order ID to proceed with handling the request.

Demo: Using Chat History and Streaming Responses

We are going to build on what we already have in our demo, and we'll bring in history and streaming responses in this one. I still have the previous application open where I have another question. But when I tried to continue the conversation, it lost track again of what we were discussing. We have seen earlier that models are stateless indeed. And by default, that is not going to be going away, let's say, by just using Semantic Kernel. We saw earlier with the OpenAI SDK that we could use a chat history, which could then be sent along with the prompt so that the context is known to the model. It is the client that is responsible for keeping track of this and sending that over to the model. It is not the model itself who's going to handle that. Let's look at some code where I'm going to bring in support for the chat history. The first part is, of course, similar. I'm building the same kernel instance. However, I'm using a different approach here. I'm going to ask for the creation of a chat completion service using kernel.GetRequiredService in IChatCompletionService, passing in the type, so IChatCompletionService, that I want. Other options exist here too. So this is a more generic approach to getting access to the services we need in our application. Next, I'm creating a ChatHistory instance, a built-in type from Semantic Kernel that we use to store the messages of the ongoing conversation. In the loop, the user can enter their message. And immediately, I add that on to the chat history using the AddUserMessage. It is marked using this method as a message that was entered by the user. Now with the updated history in place, the next line calls the chatCompletionService.GetMessageContentAsync, passing in the chat history. This method sends the entire conversation history to the model, enabling it to generate a response that takes previous messages into account. The response from the model is then displayed to the user and is also added to the chat history again. So the assistant messages are tracked alongside the user's input. Okay, let's run this and try that same conversation again. Okay, so here's my first question again, and I can now, without giving it any extra further context, ask a new question. But indeed, the model knows what we were talking about without the model, in fact, remembering just a thing. It's all passed in by the client app. Now while this was quick, we saw earlier also that it's a better user experience if the user doesn't have to wait for the entire response to be ready before just seeing anything. We fix that with a streaming response, and that too is supported by Semantic Kernel. In this SimpleContentStreaming method, I'm adding exactly that. And the bulk of the code is the same. And what is different

looks still very similar to what we saw with OpenAI. I've added manually in this part a few messages to the history initially. And then we will send over the chat request, this time using the GetStreamingChatMessageContentsAsync, which I call on the chatCompletionService instance again. So now instead of receiving the full response in one go, we get the response as a stream of StreamingChatMessageContent updates. The await foreach loop allows us to receive these updates in real time and display them in the console as they arrive, again, creating a more dynamic streaming effect again.

Demo: Working with Settings

Models have different settings that we can play with. So let's see how we can, through the use of Semantic Kernel, work with these to tweak the response we're getting back. I have one more method in this class, and that is this one here, ChangeOpenAISettings, which, not surprisingly, will play around with the settings. This first part is pretty much the same. You've already seen that. Next, to change the settings using Semantic Kernel, we use KernelArguments, basically a dictionary of key value pair arguments for the model to use. In this first instance here, I'm configuring specific values for the MaxTokens and the Temperature. Here, MaxToxens is set to 500, which gives the model enough room to produce a detailed story. The temperature setting is the key focus, controlling the creativity and the randomness again of the model responses. I let it create a story with these settings. Given the low temperature setting, this prompt is likely to create a structured factual response, let's say. We'll see the result in a second. In his second run, I am changing the temperature to 1, which will create a more varied and imaginative response. All right, time to see these two responses next to each other, or I should say below each other because it'll be in a console window. So here are the two results. I'll leave it up to you to read what the model came up with this time and get a feel of the differences in style that are caused by the different settings. But what is key here is that we can actually also change those settings easily through Semantic Kernel.

Demo: Generating Images with Semantic Kernel

We have seen, of course, that OpenAI can generate images using the DALL-E 3 model. But that too can be done using the Semantic Kernel abstraction layer, making it, again, simpler. Let's take a look I have in the ImageGeneration class this one single method, GenerateBasicImage, where you can already see that I have used a prompt we have used before. But what sits around the prompt is more interesting for us now. First, I create a kernel. But this time, I'm adding support for the OpenAI text-to-image service, so DALL-E, using AddOpenAITextToImage The passed-in model name will be DALL-E 3, and the key, of course, is still the same one. Similar to how we retrieved the chat

completion service, we're now accessing the TextToImageService using these lines here. Then, the method calls the imageService.GenerateImageAsync, passing in the prompt along with the designed dimensions to generate an image with the specified details and size. This asynchronous call returns a URL for the generated image, which is printed to the console. Indeed, I am now this time not downloading the bytes. I'm simply going to show the URL of the generated image. Let's try this out and see if we can get an image generated. The Semantic Kernel generated a link to the image, which I can now copy to the browser. And there, indeed we see the generated image, which looks very similar to our earlier image. Of course, that's quite logical as it is the same model and the same prompt that created it.

Working with Plugins

As we have learned, models can only work with the information they were trained on, which often includes a cut-off date. That means also that it can't retrieve live weather information, book an appointment, or access company-specific databases. You've seen earlier when we looked at OpenAI that function calling is a way to bridge that gap. Using function calling, we create a set of functions, tools essentially, to allow it to interact with external functions. So in the real world, outside of the LLM, we have see how we can add this through the OpenAI .NET library. And yes, it works, but it was quite a lot of ceremony. Using Semantic Kernel, this becomes a lot easier. Before we create functions with Semantic Kernel, let's look at the diagram which explains the flow of how this process now works. The first thing Semantic Kernel will do is serializing the functions. This means that the potential functions the AI could call are packaged in a way that the model can understand. These are the available tools or APIs the model can use to fulfill the request. Once the functions are serialized, the message is sent to the model. So in the next step, the model will process the input. This is where the AI analyzes the user request and determines if it can handle the request with its internal knowledge or whether it needs to call an external function for help. If the model decides a function is needed, it moves on to handling the function response. Here the model extracts the function name and parameters needed to execute the action. For example, if the user asks for stock prices, the function name might be GetStockPrice, and the parameters could include a stock symbol. Once the function parameters are identified, the kernel then invokes the function. This is where the model steps back, and functions take over really. The external API or service runs the function, whether it's retrieving data from a database or calling a third-party API. And after the function completes its job, the function result is returned to the model. The function's output like a stock price or a database query result is sent back. Finally, the kernel handles the chat response and sends the information back to the person. The model takes the function's output, combines it with its internal knowledge, and provides a full response to the user. The function or functions we want to expose this way to the

model are wrapped in plugins, which I touched upon already briefly. There are multiple ways of creating plugins, but let's here focus on adding one here using native code. Here we have a regular C# method, GetCurrentDateAndTime, that will run from within our app. But notice, I've added the KernelFunction attribute onto it, which will ensure this gets exposed to the kernel. It's vital that we pass in information about what the function can do so that the model, when it receives your input, knows it can look at the function we have defined. Semantic Kernel will do its part here. But we also need to include the description attribute here too. Just adding the attribute won't do anything just yet. We still need to register our plugin with the kernel. There are several options here, but I'm using the simplest one, the input plugin from type, passing in that type that contains the plugin, so the functions we want to expose.

Demo: Working with Plugins

We have seen how function calling works with OpenAI's SDK. And while it definitely works, it's a lot of work. It's a lot of manual code we need to write. In this demo, we'll see how easy it is to work with plugins in Semantic Kernel to do the exact same thing, as said, in an easier way. And examples of working with plugins can be found in this class, UsingPlugins. Let's start with this simple example, which will ask our model how many days there are to go until it's Christmas. A model will be smart enough to know when you ask it that question that it needs to start with knowing what is the current date. However, since models don't know what the date is, since that changes all the time, we need to provide a model with a way to figure out the current date. So that's what I'm going to do is I'm going to extend the kernel's capabilities by importing a custom plugin called TimePlugin. That's a custom type I have created as part of this application. I'll show it in just a second. So by calling kernel.InputPluginFromType in TimePlugin, we add the TimePlugin class as a new functionality within the kernel. Here is the TimePlugin class, which contains a method called GetCurrentDateAndTime. And this method is decorated with the KernelFunction attribute, marking it as a KernelFunction that Semantic Kernel can call when it's relevant to the prompt. In other words, when we go to the model for the first time, information about the fact that this function exists and can be invoked by the model is sent along. That's all done by Semantic Kernel. If the model later on indicates that this method is needed to be invoked, it will again be Semantic Kernel that handles the invocation of the method and sends over the response of this method. The Description attribute provides a brief explanation of what the function does, which is very much needed. The description is useful for the kernel when it's deciding which functions might be relevant for a given question. The function itself calls the DateTime.UtcNow to get the current date and time. Back in the GetDaysUntilChristmas method, we then define the OpenAIPromptExecutionSettings, setting ToolCallBehavior to ToolCallBehavior.AutoInvokeKernelFunctions. This setting instructs the kernel to

automatically invoke available functions like GetCurrentDateAndTime when they are relevant to the prompt, therefore streamlining the process for the user. And finally, we send a prompt to the kernel saying how many days are there until Christmas this year? And so by invoking kernel.InvokePromptAsync with the prompt and settings, the kernel can respond based on both the model's capabilities and the additional date function. All right, I hope you understand. Let's run this. And there we go. In one go, we get the response. So it first looks at when Christmas is, and then it looked at the current date, and it therefore calculated how many days there are to go until Christmas this year. Now behind the scenes though, the function was invoked. I have a breakpoint here. And I'll run things again, and I'll show you that the breakpoint actually gets hit. Indeed, our method locally is being evoked. If I compare the approach that we can use with Semantic Kernel to the OpenAI SDK with the tool creation, this is much cleaner and simpler to use, using just a few simple attributes. Now we aren't limited, of course, to just using one simple plugin. And when adding multiple ones, the kernel can use these to make the responses of the model even better. Again, remember, that the kernel is the manager that sits in between and takes a lot of the work out of our hands. In this method here, I have added two plugins. The second plugin class, WeatherPlugin, is a very simplified weather tool. The only method it has, GetWeather, is marked again with the KernelFunction attribute and also has a description. Now once more now, before I forget, plugin classes can contain more than one kernel function, just to be clear. And this method here has two parameters, for which it's important that the descriptions are provided as well. In the end, it is the model that needs to provide a value for these. So it's important that it knows what needs to be provided. Now my Weather method is very simple, as you can see, and it is going to return only the weather for Brussels. Now back in the method, I'm calling the InputPluginFromType multiple times, once for each class that contains our Kernel function that is. The rest of the code is similar. I have a small chat loop in here with history support again. First, I'm going to ask, what is the weather like today? That will trigger the invocation of the TimePlugin since the model needs to figure out based on my question, what is the date? Now it comes back to me and asks me to provide a location. So let's add the Brussels here, and there we go. Our second plugin is now invoked, and we get the response. Perfect. Once again, what we have seen here is similar to what we had with function calling in OpenAI directly. However, the way that this is done in Semantic Kernel is a lot simpler. What I have shown you in this module is just a small part of what Semantic Kernel is capable of. In fact, it is such an amazing library that we have a separate course in the library on Pluralsight called Building AI Applications with Semantic Kernel and C#, which will be available in late 2024.

Working with Semantic Kernel from an ASP.NET Core Blazor Application

Demo: Adding Semantic Kernel

Semantic Kernel can do quite a lot. With what I have covered in the previous module, that should be clear. Now it's time to make things more real world again. So in this module, we'll spend time adding Semantic Kernel's features to our Blazor application once again. So, no time to waste. In this first demo, I'm going to configure the application and thus bring in the right package and register things correctly so that in the upcoming demos, we can use Semantic Kernel in our application. You've seen before that adding Semantic Kernel is basically nothing more than adding the correct package. And of course, that hasn't changed. Here's the project file where I now added the latest version of Semantic Kernel at the time of the recording of this course. Since we are again in a web context or inside the Blazor application, it's again easier to register everything in one location, so in the Program class. Here is the code that should already be rather familiar to you after what we've done in the previous module. I'm using AddKernel here now to register Semantic Kernel with the ASP.NET Core dependency injection system so that we can later on elsewhere in the application inject it easily. After that, I am chaining two AddOpenAI calls, which are configuring chat completion and text-to-image generation. Since this is a web app again, I'm using settings stored in development mode inside the secrets.json file. And that contains values for the text model name, the image model name, and the OPEN API key. By again setting these values here, we have specified to Semantic Kernel that it has to use the OpenAI's models. And I think with that, all that needs to be done to configure our app for use with Semantic Kernel is ready.

Demo: Generating Text and Images

Earlier in the course, we have created the Pie Add page with capabilities to generate text and an image based on the name of the pie that was entered by the user. We've done this so far using the OpenAI SDK. But this can, of course, also be done using Semantic Kernel, which is what we'll do here in this demo. Just as a small refresher, here is the running Pie Add page. In fact, this time, it is powered by Semantic Kernel, but you don't see that. It has the same functionality. I've entered the name, and the content here is being generated as we can see here. Also the image can be generated in the same way. In the code, things have, of course, changed since I have now replaced the OpenAI SDK with code using Semantic Kernel. We can see that I'm injecting the kernel here through the ASP.NET Core dependency injection system since we registered that. Next, in the OnInitialized, I'm going to use that kernel instance to get inside this component a ChatCompletionService and a TextToImageService instance, which I'll need to get the desired functionality . In the GenerateShortDescription, which is, of course, called when I clicked on that button in UI, we're now using Semantic Kernel, the prompt we've already

used. Here, I'm limiting the length again to 20 words. I then create a chat history, and we continue to use the ChatCompletionService. And on that, I invoke the GetStreamingChatMessageContentsAsync method, which will again stream the response coming back from the model so that, again, we don't have to wait for the entire content block to be shown for each piece of streamed content received represented by a chatUpdate.Content we appended to the Pie.ShortDescription. This will build up the full description as each part of the content arives. Additionally, StateHasChanged is called after each update, which signals to Blazor that the state has changed and that the UI needs a refresh. As you can see, using Semantic Kernel for this task is very simple. In the GenerateLongDescription, I am doing pretty much the same thing but with a different prompt. Finally, I've also replaced the code to generate an image, now using Semantic Kernel. This code we have already covered in the previous module, so let's go over it quickly. I'm using this time the textToImageService, and I invoke the GenerateImageAsync method. And since with Semantic Kernel this generates a URL to the generated image, I need to include some code, regular .NET code that is, to download the image to a local file on the server. And when that is done, we can update the ImageUrl of the pie, which can then be saved. So there we go. We've now seen that using Semantic Kernel for these simple tasks is, well, pretty simple.

Demo: Using Plugins

Using OpenAI, we have used function calling to create a sales bot. That was quite a lot of code. In this demo, I'll show you how we can use plugins to create again the same functionality. At the start of the demo, here is the running sales bot again where I can enter a question about our sales data in this field. Of course, OpenAI doesn't know hopefully about our sales number. So it used a local function that was invoked, and the result of that call is sent back to OpenAI to enhance its answer. And that's what we see here. In the code, I've now created a version that uses Semantic Kernel plugins. Here's the code where I'm again injecting now a kernel. In the OnInitialized, part of the magic is already happening. I get a ChatCompletionService instance like we've done before. But I also import a plugin into the kernel using the ImportPluginFromType, saying that I want the SalesInformationPlugin to be added. That's a type that, for the sake of simplicity, I've added in this file, so down here. That's my plugin class, and it contains a KernelFunction. Now note first that this class also uses dependency injection to get access to the orderDataService, which is injected here. This is regular .NET code. We can do basically what we want from here. In the KernelFunction, I've also added again a description, and it's quite an extended one again. The code itself will just retrieve the number of orders for a given month of a given year. Back here in the OnInitialized, I say that the ToolCallBehavior is, again, set to Automatic. And when the user enters a question and clicks on the button, I'm evoking this code where I first add a system

message, basically setting the scene. User's question is also added to the history. And then we start the process of talking with the AI model with just this line here, GetChatMessageFromAsync, passing in the chatHistory, the settings, and the kernel, which knows the plugin. Do you see that whole switch statement that we had with the OpenAI SDK? Well, I don't. All of that, so the back and forth between my application and the model, is all done for us. So it definitely becomes a lot simpler using Semantic Kernel.